학습 교안

SQLD

SQL 개발자 자격 대비

1과목 데이터 모델링의 이해 · 2과목 SQL 기본 및 활용
이론 30개 절 · SQL 예제 79개 (실제 실행 결과) · 문제 193개 · SQL 실습 15문제

강사 김대철

학습 앱: https://sinaboro.github.io/sqld-learning-app-/

이 교안의 사용법

이 교안은 SQLD(SQL 개발자) 시험을 처음 준비하는 사람을 위해 만들었어요. 이론서(1~5장)는 **학습 목표 → 설명과 SQL 예제 → 확인 문제**, 기출공략집(6장)은 대표 유형 30선과 변형문제 2회, 보너스(7장)는 고난이도 문제와 SQL 실습 문제로 되어 있어요. SQL 예제는 모두 예제 DB(EMP·DEPT·SALGRADE, 부록)에서 실제로 실행한 결과예요.

표시	뜻
SQL	예제 DB 에서 실행해 볼 SQL (파란 줄)
실행 결과	그 SQL 을 실제로 실행한 결과 표 (주황 줄)
SQL (오라클 문법)	ROLLUP, CONNECT BY, PIVOT 처럼 오라클 전용 문법. 바로 아래에 같은 결과를 내는 표준 SQL 을 함께 실었어요
TIP / 주의	알아 두면 좋은 내용 / 자주 하는 실수·시험 함정
확인 문제	절 끝의 문제. 정답과 해설은 맨 뒤에 있어요

학습 앱 교안과 똑같은 내용의 학습 앱이 있어요. https://sinaboro.github.io/sqld-learning-app-/ 을 열면 SQL 을 고쳐 가며 바로 실행하고, 문제를 풀고, 실전 모의고사 01~05회(50문항·90분·과락 채점)와 SQL 실습 채점을 할 수 있어요. 카카오톡으로 링크를 받아 홈 화면에 설치하면 앱처럼 쓸 수 있습니다.

공부 순서 추천

① 이론을 읽고 → ② SQL 결과를 **먼저 예상한 뒤** 실행 결과와 비교하고 → ③ 절 끝 확인 문제를 풀고 → ④ 6장 대표 유형·변형문제 → ⑤ 앱의 실전 모의고사로 마무리하세요. 결과 행 수를 묻는 문제가 많으니 부록의 예제 DB 데이터를 옆에 두고 공부하면 좋아요.

목차

이 교안의 사용법 2

01장 [1과목] 데이터 모델링 5

01.1 데이터 모델의 이해 5

01.2 엔터티 6

01.3 속성 7

01.4 관계 8

01.5 식별자 9

02장 [1과목] 데이터 모델과 SQL 11

02.1 정규화 11

02.2 관계와 조인의 이해 12

02.3 모델이 표현하는 트랜잭션의 이해 13

02.4 NULL 속성의 이해 14

02.5 본질식별자 vs 인조식별자 16

03장 [2과목] SQL 기본 18

03.1 관계형 데이터베이스 개요 18

03.2 SELECT 문 19

03.3 함수(Function) 20

03.4 WHERE 절 23

03.5 GROUP BY, HAVING 절 25

03.6 ORDER BY 절 27

03.7 조인 28

03.8 표준 조인 30

04장 [2과목] SQL 활용 32

04.1 서브쿼리 32

04.2 집합 연산자 34

04.3 그룹 함수 35

04.4 윈도우 함수 37

04.5 TOP-N 쿼리 40

04.6 계층형 질의와 셀프 조인 41

04.7 PIVOT 절과 UNPIVOT 절 43

04.8 정규 표현식 44

05장 [2과목] 관리 구문 46

05.1 DML 46

05.2 TCL 48

05.3 DDL 49

05.4 DCL 52

06장 기출공략집 54

06.0 시험 안내와 활용법 54

06.1 대표 유형 30선 54

06.2 기출 변형문제 01회 59

06.3 기출 변형문제 02회 62

07장 보너스 합격자료집 68

07.1 SQLD 고난이도 예상문제 68

07.2 SQL 실습 문제 74

SQL 실습 문제 정답 76

부록 예제 데이터베이스 81

정답과 해설 82

CHAPTER 01

[1과목] 데이터 모델링

학습 목표

- 데이터 모델링의 특징·관점·단계와 3층 스키마를 설명할 수 있다.
- 엔터티의 특징과 분류를 구분한다.
- 속성의 분류(기본·설계·파생)와 도메인을 안다.
- 관계의 차수와 선택성을 읽을 수 있다.
- 주식별자의 4가지 특징과 식별·비식별 관계를 구분한다.

01.1 데이터 모델의 이해

데이터 모델링은 현실 세계의 업무를 데이터베이스로 옮기기 위해 **정보를 구조화해 표현하는 과정**이에요. 건물을 짓기 전에 설계도를 그리는 것과 같아요.

모델링의 3가지 특징

특징	뜻
추상화	현실의 복잡한 내용을 일정한 형식에 맞춰 표현
단순화	누구나 이해하기 쉽게 약속된 표기법으로 표현
명확화	애매함을 없애고 한 가지 뜻으로 정확하게 표현

모델링의 3가지 관점

- **데이터 관점**: 업무가 **어떤 데이터**를 다루는가 (What, Data)
- **프로세스 관점**: 업무가 **무엇을 하는가** (How, Process)
- **데이터와 프로세스의 상관 관점**: 업무 처리가 데이터에 **어떤 영향**을 주는가 (Interaction)

모델링의 3단계

단계	하는 일	특징
개념적 모델링	핵심 엔터티와 관계를 찾아 ERD 작성	추상화 수준이 가장 높음, 업무 중심
논리적 모델링	키·속성·관계를 정확히 정의, **정규화** 수행	재사용성이 높음, DBMS 에 독립적
물리적 모델링	실제 DB 에 맞게 테이블·인덱스·저장 구조 설계	성능·보안·저장 공간 고려

데이터 독립성과 3층 스키마

데이터 구조가 바뀌어도 응용 프로그램이 영향을 덜 받도록 **3층 스키마**로 나눠요.

스키마	관점	설명
외부 스키마	사용자	사용자·프로그램마다 보는 데이터 (뷰)
개념 스키마	통합	모든 사용자의 데이터를 통합한 전체 구조
내부 스키마	물리 저장	실제 저장 방식 (파일, 인덱스)

- **논리적 독립성**: 개념 스키마가 바뀌어도 외부 스키마는 영향 없음
- **물리적 독립성**: 내부 스키마가 바뀌어도 개념·외부 스키마는 영향 없음

ERD 와 좋은 데이터 모델

ERD(Entity Relationship Diagram)는 **엔터티·속성·관계**로 데이터 구조를 그린 그림이에요. 그리는 순서는 ① 엔터티 도출 → ② 엔터티 배치 → ③ 관계 설정 → ④ 관계명 기술 → ⑤ 관계 참여도(차수) → ⑥ 관계 필수 여부 표시입니다.

```
화면
 [부서 DEPT]                    [사원 EMP]
 # 부서번호    ──────────<    # 사원번호
 * 부서명        1 : N         * 이름
 o 위치                       o 부서번호(FK)
```

주의 모델링할 때 피해야 할 것: **중복**(같은 데이터를 여러 곳에 저장), **비유연성**(작은 업무 변화에도 모델이 크게 바뀜), **비일관성**(데이터끼리 서로 맞지 않음).

확인 문제

Q1. 데이터 모델링의 특징이 아닌 것은?

① 추상화
② 단순화
③ 명확화
④ 구체화

Q2. 정규화를 수행하고 DBMS 에 독립적인 모델을 만드는 단계는?

① 개념적 모델링
② 논리적 모델링
③ 물리적 모델링
④ 프로세스 모델링

Q3. 내부 스키마가 바뀌어도 개념 스키마가 영향을 받지 않는 성질은?

① 논리적 독립성
② 물리적 독립성
③ 무결성
④ 일관성

Q4. 사용자나 응용 프로그램이 보는 관점의 스키마는?

① 외부 스키마
② 개념 스키마
③ 내부 스키마
④ 물리 스키마

01.2 엔터티

엔터티(Entity)는 업무에 필요하고 **저장해서 관리해야 하는 대상**이에요. 사원, 부서, 주문처럼 명사로 표현되고, 테이블이 됩니다. 엔터티 안의 개별 데이터(사원 SMITH 한 명)를 **인스턴스**라고 해요.

엔터티의 특징

1. 반드시 해당 **업무에서 필요하고 관리하고자 하는** 정보여야 한다.
2. **유일한 식별자**로 구분할 수 있어야 한다.
3. **인스턴스가 2개 이상**이어야 한다. (하나뿐이면 엔터티가 아님)
4. 업무 **프로세스에 이용**되어야 한다.
5. 반드시 **속성**을 가져야 한다.
6. 다른 엔터티와 **최소 1개 이상의 관계**가 있어야 한다. (코드·통계성 엔터티는 예외 가능)

엔터티의 분류

기준	분류	예
유무형	유형 엔터티 (물리적 형태가 있음)	사원, 상품, 강사
	개념 엔터티 (형태 없는 개념)	조직, 보험상품
	사건 엔터티 (업무 수행 중 발생)	주문, 청구, 수강신청
발생 시점	기본 엔터티 (키 엔터티, 독립적으로 생성)	사원, 부서, 고객, 상품
	중심 엔터티 (기본에서 발생, 업무의 중심)	계약, 주문, 매출
	행위 엔터티 (2개 이상 엔터티에서 발생, 자주 바뀜)	주문 이력, 접속 이력

TIP 엔터티 이름은 현업에서 쓰는 용어로, **단수 명사**로, 약어 없이, 유일하게 짓습니다.

확인 문제

Q1. 엔터티의 특징으로 틀린 것은?

① 업무에서 관리해야 하는 정보다
② 유일한 식별자로 구분된다
③ 인스턴스가 1개만 있어도 된다
④ 업무 프로세스에 이용된다

Q2. 업무 수행 중에 발생하는 '주문', '청구' 같은 엔터티는?

① 유형 엔터티
② 개념 엔터티
③ 사건 엔터티
④ 기본 엔터티

Q3. 발생 시점에 따른 분류 중 독립적으로 생성되는 엔터티는?

① 기본 엔터티
② 중심 엔터티
③ 행위 엔터티
④ 사건 엔터티

01.3 속성

속성(Attribute)은 엔터티가 가진 **더 이상 나눌 수 없는 최소 단위의 정보**예요. 테이블의 칼럼이 됩니다. 한 속성은 한 인스턴스에서 **하나의 값만** 가져야 해요.

특성에 따른 분류

분류	뜻	예
기본 속성	업무에서 원래 가진 속성	이름, 급여, 입사일
설계 속성	업무에 없지만 설계하며 만든 속성	사원번호, 상태코드
파생 속성	다른 속성에서 계산해 낸 속성	합계, 평균, 나이

TIP 파생 속성은 원래 값이 바뀌면 함께 고쳐야 해서 **데이터 정합성 문제**가 생길 수 있어요. 꼭 필요할 때만 씁니다.

구성 방식·값의 개수에 따른 분류

- **PK(기본키) 속성**: 인스턴스를 식별 / **FK(외래키) 속성**: 다른 엔터티와의 관계 / **일반 속성**: 나머지

- **단순 속성**: 더 나눌 수 없음 (나이) / **복합 속성**: 나눌 수 있음 (주소 → 시·구·동)
- **단일값 속성**: 값이 하나 (주민번호) / **다중값 속성**: 값이 여럿 (전화번호 여러 개) → 별도 엔터티로 분리

도메인

속성이 가질 수 있는 **값의 범위**를 도메인이라고 해요. (자료형, 길이, 허용 값, 기본값, 제약조건) 예: 급여는 0 이상 정수, 성별은 'M' 또는 'F'.

```
SQL
-- 테이블의 칼럼(속성) 확인: EMP 엔터티의 속성들
SELECT EMPNO, ENAME, JOB, SAL, COMM FROM EMP WHERE DEPTNO = 10;
```

```
실행 결과
EMPNO ¦ ENAME  ¦ JOB       ¦ SAL  ¦ COMM
------+--------+-----------+------+-----
7782  ¦ CLARK  ¦ MANAGER   ¦ 2450 ¦ NULL
7839  ¦ KING   ¦ PRESIDENT ¦ 5000 ¦ NULL
7934  ¦ MILLER ¦ CLERK     ¦ 1300 ¦ NULL
(3개 행)
```

확인 문제

Q1. 다른 속성에서 계산해 만든 '주문 합계 금액' 같은 속성은?

① 기본 속성
② 설계 속성
③ 파생 속성
④ 복합 속성

Q2. 속성이 가질 수 있는 값의 범위를 무엇이라 하나요?

① 인스턴스
② 도메인
③ 카디널리티
④ 스키마

Q3. 업무에는 없지만 설계 과정에서 만든 '사원번호' 같은 속성은?

① 기본 속성
② 설계 속성
③ 파생 속성
④ 다중값 속성

01.4 관계

관계(Relationship)는 엔터티의 인스턴스끼리 **논리적으로 연결된 것**이에요. "부서는 사원을 **포함한다**", "고객이 상품을 **주문한다**"처럼 표현합니다.

관계의 종류

- **존재 관계**: 존재 자체로 연결 (부서와 사원 — 사원은 부서에 소속)
- **행위 관계**: 어떤 행위가 있어야 연결 (고객이 주문을 **한다**)

관계 표기법 3요소

요소	뜻	예
관계명	관계의 이름 (양쪽 방향)	포함한다 / 소속된다

요소	뜻	예
관계 차수 (카디널리티)	참여하는 수	1:1, 1:M, M:N
관계 선택사양 (선택성)	반드시 참여하는가	필수 / 선택 (IE 표기의 ○)

화면
```
부서 ┼────────○< 사원        → 한 부서에 사원이 0명 이상(선택), 사원은 반드시 한 부서(필수)
(1)                (M)
```

주의 M:N 관계는 관계형 DB 에서 바로 구현할 수 없어요. 두 엔터티 사이에 **교차(연관) 엔터티**를 만들어 1:M + M:1 로 풀어냅니다. (학생 M:N 과목 → 학생 1:M 수강신청 M:1 과목)

관계 체크사항

두 엔터티 사이에 **연관 규칙**이 있는가, **정보의 조합**이 발생하는가, 업무 기술서에 관계 **연결을 가능하게 하는 동사**가 있는가, **업무 기술서에 관계 연결에 대한 규칙**이 서술되어 있는가를 확인해요.

확인 문제

Q1. 관계 표기법의 3요소가 아닌 것은?

① 관계명
② 관계 차수
③ 관계 선택사양
④ 관계 정규화

Q2. M:N 관계를 관계형 DB 로 구현하는 방법은?

① 그대로 두 테이블로 구현한다
② 교차 엔터티를 만들어 1:M 관계 두 개로 푼다
③ 한 테이블로 합친다
④ 외래키를 두 테이블 모두에 넣는다

Q3. 고객이 상품을 '주문한다' 처럼 행위가 있어야 생기는 관계는?

① 존재 관계
② 행위 관계
③ 식별 관계
④ 상호배타 관계

01.5 식별자

식별자(Identifier)는 엔터티의 인스턴스를 **하나씩 구분해 주는 속성**이에요. 대표 식별자가 주식별자(테이블의 기본키, PK)입니다.

주식별자의 4가지 특징

특징	뜻
유일성	모든 인스턴스를 **유일하게** 구분한다
최소성	유일성을 만족하는 **최소한의 속성**으로 구성한다
불변성	한 번 정하면 값이 **바뀌지 않는다**
존재성	반드시 값이 **있어야** 한다 (NULL 불가)

식별자의 분류

기준	분류
대표성	주식별자 / 보조식별자 (유일하지만 대표가 아님, 참조 관계 연결에 안 씀)
생성 여부	내부식별자 (엔터티 안에서 생성) / 외부식별자 (다른 엔터티에서 온 FK)
속성 수	단일식별자 (속성 1개) / 복합식별자 (속성 2개 이상)
대체 여부	본질식별자 (업무에서 원래 쓰는 값) / 인조식별자 (일련번호처럼 인위적으로 만듦)

식별자 관계 vs 비식별자 관계

구분	식별자 관계	비식별자 관계
부모의 주식별자가	자식의 **주식별자 일부**가 됨	자식의 **일반 속성(FK)**이 됨
표기 (IE)	실선	점선
연결 강도	강한 관계	약한 관계
장점	조인 없이 부모 키로 바로 조회	자식 PK 가 단순, 유연함
단점	자식으로 갈수록 PK 속성이 늘어남	부모 정보가 필요하면 조인이 늘어남

TIP **비식별자 관계를 고르는 경우**: 부모 없이 자식이 생길 수 있을 때(약한 관계), 자식의 주식별자를 독립적으로 구성하고 싶을 때, 식별자 관계로 PK 속성이 너무 많아질 때.

확인 문제

Q1. 주식별자의 특징이 아닌 것은?

① 유일성
② 최소성
③ 불변성
④ 다양성

Q2. 부모 엔터티의 주식별자가 자식 엔터티의 일반 속성이 되는 관계는?

① 식별자 관계
② 비식별자 관계
③ 상호배타 관계
④ 순환 관계

Q3. 다른 엔터티와의 관계로 받아온 식별자는?

① 내부식별자
② 외부식별자
③ 보조식별자
④ 본질식별자

Q4. '주식별자는 반드시 값이 있어야 한다'는 특징은?

① 유일성
② 최소성
③ 불변성
④ 존재성

CHAPTER 02

[1과목] 데이터 모델과 SQL

학습 목표

- 함수적 종속을 이해하고 1·2·3정규형과 BCNF 를 구분한다.
- 반정규화의 목적과 기법을 안다.
- 모델의 관계가 SQL 조인으로 어떻게 표현되는지 안다.
- 트랜잭션의 4가지 성질(ACID)과 모델에서의 표현을 안다.
- NULL 의 연산·비교·집계 규칙을 정확히 안다.
- 본질식별자와 인조식별자의 장단점을 비교한다.

02.1 정규화

정규화는 데이터의 **중복을 없애고 이상현상을 막기 위해** 테이블을 알맞게 쪼개는 과정이에요. 중복이 있으면 아래 같은 **이상현상(Anomaly)**이 생겨요.

이상현상	예 (수강 테이블에 학생 주소가 함께 있을 때)
삽입 이상	과목을 신청하지 않은 학생은 주소를 저장할 수 없다
갱신 이상	주소가 바뀌면 여러 행을 다 고쳐야 하고, 하나라도 빠지면 불일치
삭제 이상	마지막 수강 기록을 지우면 학생 주소까지 사라진다

함수적 종속

`A → B` 는 "A 값을 알면 B 값이 하나로 정해진다"는 뜻이에요. 이때 A 를 **결정자**, B 를 **종속자**라고 합니다. (사원번호 → 이름)

정규형 단계

정규형	조건	해결 방법
1정규형 (1NF)	모든 속성이 **원자값**(하나의 값)	다중값·반복 속성을 행이나 별도 테이블로 분리
2정규형 (2NF)	1NF + **부분 함수 종속 제거** (복합 PK 의 일부에만 종속된 속성이 없음)	일부 키에만 종속된 속성을 분리
3정규형 (3NF)	2NF + **이행 함수 종속 제거** (A→B, B→C 일 때 A→C 없앰)	키가 아닌 속성에 종속된 속성을 분리
BCNF	3NF + **모든 결정자가 후보키**	후보키가 아닌 결정자를 분리

화면

```
[수강] (학번, 과목코드) → 성적          ← 완전 함수 종속 OK
       학번 → 학생이름                  ← 부분 함수 종속 → 2NF 위반 → [학생] 분리
[사원] 사원번호 → 부서번호 → 부서명      ← 이행 함수 종속 → 3NF 위반 → [부서] 분리
```

TIP 우리 예제 DB 도 정규화되어 있어요. 사원(EMP)에는 부서번호만 있고, 부서명은 부서(DEPT)에 따로 있죠. 이행 종속(사원번호 → 부서번호 → 부서명)을 없앤 3정규형입니다.

반정규화 (역정규화)

정규화된 모델에서 **조회 성능**을 높이려고 일부러 중복을 허용하거나 테이블을 합치는 것이에요. 단, **데이터 무결성이 깨질 위험**이 있으니 꼭 필요할 때만 합니다.

대상	기법
테이블	테이블 병합(1:1, 1:M, 슈퍼/서브타입), 테이블 분할(수직·수평), 중복 테이블 추가(통계·이력·부분 테이블)
칼럼	중복 칼럼 추가, 파생 칼럼 추가, 이력 테이블에 칼럼 추가, PK 에 의한 칼럼 추가, 응용 오작동 대비 칼럼 추가
관계	중복 관계 추가

TIP 반정규화 절차: ① 반정규화 대상 조사 (범위·빈도·통계) → ② **다른 방법 먼저 검토** (뷰, 클러스터링, 인덱스, 응용 프로그램) → ③ 반정규화 적용.

확인 문제

Q1. 복합 기본키의 일부에만 종속된 속성을 분리하는 정규형은?

① 1정규형
② 2정규형
③ 3정규형
④ BCNF

Q2. 사원번호 → 부서번호 → 부서명 같은 종속을 없애는 정규형은?

① 1정규형
② 2정규형
③ 3정규형
④ 4정규형

Q3. 반정규화에 대한 설명으로 옳은 것은?

① 데이터 무결성이 항상 높아진다
② 조회 성능을 위해 중복을 일부 허용한다
③ 정규화보다 반드시 먼저 한다
④ 인덱스·뷰 검토 없이 바로 적용한다

Q4. 마지막 수강 기록을 지웠더니 학생 정보까지 사라지는 현상은?

① 삽입 이상
② 갱신 이상
③ 삭제 이상
④ 조회 이상

02.2 관계와 조인의 이해

모델에서 엔터티 사이의 **관계**는 SQL 에서 **조인(JOIN)**으로 다시 이어 붙여 조회해요. 부서(1)와 사원(M)의 관계는 사원의 FK(DEPTNO)와 부서의 PK(DEPTNO)를 같게 놓고 조인합니다.

SQL
```
-- 관계(부서 1 : 사원 M)를 조인으로 표현: 사원 이름과 부서 이름 함께 보기
SELECT E.ENAME, E.DEPTNO, D.DNAME
FROM EMP E, DEPT D
WHERE E.DEPTNO = D.DEPTNO        -- FK = PK 로 관계를 잇는다
  AND E.DEPTNO = 10;
```

실행 결과
```
ENAME  ¦ DEPTNO ¦ DNAME
-------+--------+-----------
CLARK  ¦ 10     ¦ ACCOUNTING
KING   ¦ 10     ¦ ACCOUNTING
MILLER ¦ 10     ¦ ACCOUNTING
(3개 행)
```

순환 관계 (자기 참조)

한 엔터티가 **자기 자신과 관계**를 맺는 것이에요. 사원의 관리자(MGR)도 사원이죠. 같은 테이블을 두 번 써서 조인합니다(셀프 조인, 2과목에서 자세히).

```
SQL
-- 사원과 그 관리자 이름
SELECT W.ENAME AS 사원, M.ENAME AS 관리자
FROM EMP W, EMP M
WHERE W.MGR = M.EMPNO AND W.DEPTNO = 20;
```

```
실행 결과
사원   ¦ 관리자
------+-------
SMITH ¦ FORD
JONES ¦ KING
SCOTT ¦ JONES
ADAMS ¦ SCOTT
FORD  ¦ JONES
(5개 행)
```

상호배타 관계

하나의 엔터티가 두 엔터티 중 **하나와만** 관계를 맺는 것이에요. 예: 주문은 개인고객 **또는** 법인고객 중 한쪽에만 연결됩니다. (ERD 에서 관계선에 호를 그려 표시)

확인 문제

Q1. 사원의 관리자도 사원인 경우처럼 자기 자신과 맺는 관계는?

① 상호배타 관계
② 순환 관계
③ 식별 관계
④ M:N 관계

Q2. 주문이 개인고객 또는 법인고객 중 하나와만 관계를 맺는 것은?

① 상호배타 관계
② 순환 관계
③ 존재 관계
④ 1:1 관계

Q3. 부서(1) : 사원(M) 관계를 조인할 때 같게 놓는 칼럼은?

① 사원의 PK 와 부서의 PK
② 사원의 FK(부서번호)와 부서의 PK(부서번호)
③ 사원 이름과 부서 이름
④ 관계명

02.3 모델이 표현하는 트랜잭션의 이해

트랜잭션은 **함께 성공하거나 함께 실패해야 하는 작업 묶음**이에요. 계좌 이체에서 출금과 입금은 둘 다 되거나 둘 다 안 되어야 하죠.

트랜잭션의 4가지 성질 (ACID)

성질	뜻
원자성 (Atomicity)	모두 실행되거나 전혀 실행되지 않는다 (All or Nothing)
일관성 (Consistency)	실행 전후 데이터베이스가 일관된 상태를 유지한다

성질	뜻
고립성 (Isolation)	실행 중인 트랜잭션은 다른 트랜잭션의 영향을 받지 않는다
지속성 (Durability)	완료(커밋)된 결과는 영구히 저장된다

모델에서 트랜잭션 읽기

주문과 주문상세처럼 **반드시 함께 생겨야 하는** 엔터티는 관계를 **필수(Mandatory)**로 표현해요. 주문만 있고 주문상세가 없는 상태가 되면 안 되므로, 두 INSERT 를 하나의 트랜잭션으로 묶어 처리합니다.

```sql
-- 부서와 그 부서의 첫 사원을 하나의 트랜잭션으로 등록
BEGIN;
INSERT INTO DEPT VALUES (50, 'MARKETING', 'SEOUL');
INSERT INTO EMP (EMPNO, ENAME, JOB, SAL, DEPTNO) VALUES (8001, 'KIM', 'MANAGER', 3000, 50);
COMMIT;
SELECT D.DNAME, E.ENAME FROM DEPT D, EMP E WHERE D.DEPTNO = E.DEPTNO AND D.DEPTNO = 50;
```

실행 결과

```
트랜잭션을 시작합니다.

1개 행이 삽입되었습니다.

1개 행이 삽입되었습니다.

커밋되었습니다.

DNAME     ¦ ENAME
----------+------
MARKETING ¦ KIM
(1개 행)
```

확인 문제

Q1. '모두 실행되거나 전혀 실행되지 않는다'는 트랜잭션 성질은?

① 원자성
② 일관성
③ 고립성
④ 지속성

Q2. 커밋된 결과가 시스템 장애가 나도 유지되는 성질은?

① 원자성
② 고립성
③ 지속성
④ 일관성

Q3. 주문과 주문상세처럼 반드시 함께 생겨야 하는 관계를 모델에서 표현하는 방법은?

① 선택 관계로 표시한다
② 필수 관계로 표시한다
③ 관계를 없앤다
④ 상호배타 관계로 표시한다

02.4 NULL 속성의 이해

NULL 은 "값이 **없음**, 아직 **모름**"을 뜻해요. 0 이나 공백 문자와 **다릅니다.** EMP 의 COMM(커미션)은 영업사원 말고는 NULL 이에요.

NULL 규칙 4가지

규칙	예	결과
NULL 과의 **산술 연산**은 NULL	1000 + NULL	NULL
NULL 과의 **비교**는 알 수 없음(UNKNOWN) → 조건이 거짓처럼 동작	COMM = NULL	행이 안 나옴
NULL 확인은 **IS NULL / IS NOT NULL**	COMM IS NULL	정상
집계 함수는 NULL 을 빼고 계산 (COUNT(*) 제외)	AVG(COMM)	NULL 아닌 4건의 평균

```sql
-- 산술 연산: COMM 이 NULL 이면 결과도 NULL
SELECT ENAME, SAL, COMM, SAL + COMM AS 합계 FROM EMP WHERE DEPTNO = 30;
```

실행 결과

```
ENAME  ┆ SAL  ┆ COMM ┆ 합계
-------+------+------+-----
ALLEN  ┆ 1600 ┆ 300  ┆ 1900
WARD   ┆ 1250 ┆ 500  ┆ 1750
MARTIN ┆ 1250 ┆ 1400 ┆ 2650
BLAKE  ┆ 2850 ┆ NULL ┆ NULL
TURNER ┆ 1500 ┆ 0    ┆ 1500
JAMES  ┆ 950  ┆ NULL ┆ NULL
(6개 행)
```

```sql
-- = NULL 은 아무 행도 못 찾는다, IS NULL 을 써야 한다
SELECT COUNT(*) AS "= NULL" FROM EMP WHERE COMM = NULL;
SELECT COUNT(*) AS "IS NULL" FROM EMP WHERE COMM IS NULL;
```

실행 결과

```
= NULL
------
0
(1개 행)

IS NULL
-------
10
(1개 행)
```

```sql
-- 집계 함수와 NULL: COUNT(*) 는 모든 행, COUNT(COMM) 은 NULL 이 아닌 행만
SELECT COUNT(*), COUNT(COMM), SUM(COMM), AVG(COMM), AVG(NVL(COMM, 0)) FROM EMP;
```

실행 결과

```
COUNT(*) ┆ COUNT(COMM) ┆ SUM(COMM) ┆ AVG(COMM) ┆ AVG(NVL(COMM, 0))
---------+-------------+-----------+-----------+------------------
14       ┆ 4           ┆ 2200      ┆ 550       ┆ 157.1429
(1개 행)
```

TIP AVG(COMM) 은 NULL 이 아닌 4명(300, 500, 1400, 0)의 평균 550, AVG(NVL(COMM, 0)) 은 14명 전체로 나눈 157.1429 예요. **NULL 을 0 으로 볼지** 에 따라 결과가 달라지니 시험에 자주 나와요.

주의 오라클에서는 빈 문자열 `''` 을 NULL 로 취급하지만, SQL Server·SQLite 등은 빈 문자열과 NULL 을 구분해요. SQLD 는 오라클 기준으로 출제됩니다.

확인 문제

Q1. SAL = 1000, COMM = NULL 일 때 SAL + COMM 의 결과는?

① 1000
② 0
③ NULL
④ 오류

Q2. COMM 이 NULL 인 행을 찾는 올바른 조건은?

① COMM = NULL
② COMM IS NULL
③ COMM == NULL
④ COMM IN (NULL)

Q3. 값이 (100, NULL, 200) 인 칼럼의 AVG 결과는?

① 100
② 150
③ NULL
④ 300

Q4. 값이 (100, NULL, 200) 인 칼럼에 대해 COUNT(*) 와 COUNT(칼럼) 은?

① 3, 3
② 3, 2
③ 2, 2
④ 2, 3

02.5 본질식별자 vs 인조식별자

- **본질식별자**: 업무에 원래 있는 값으로 만든 식별자 (주문번호 + 상품번호)
- **인조식별자**: 일련번호처럼 **인위적으로 만든** 식별자 (주문상세번호 1, 2, 3 ...)

구분	본질식별자	인조식별자
장점	업무 의미가 분명, 중복 데이터 방지	PK 가 단순, 조인·관리가 쉬움
단점	여러 속성으로 PK 가 복잡해질 수 있음	**중복 데이터 발생 가능**, 불필요한 인덱스 추가

SQL
```
-- 인조식별자만 PK 로 쓰면 같은 내용이 중복으로 들어가도 막지 못한다
CREATE TABLE ORDER_ITEM (ITEM_ID INTEGER PRIMARY KEY, ORDER_NO INTEGER, PROD_NO INTEGER, QTY INTEGER);
INSERT INTO ORDER_ITEM VALUES (1, 100, 7, 2);
INSERT INTO ORDER_ITEM VALUES (2, 100, 7, 2);   -- 같은 주문·같은 상품이 또 들어감
SELECT * FROM ORDER_ITEM;
```

실행 결과
```
생성되었습니다.

1개 행이 삽입되었습니다.

1개 행이 삽입되었습니다.

ITEM_ID ¦ ORDER_NO ¦ PROD_NO ¦ QTY
--------+----------+---------+----
1       ¦ 100      ¦ 7       ¦ 2
2       ¦ 100      ¦ 7       ¦ 2
(2개 행)
```

SQL (오류가 나는 것이 정상인 예)

```
-- 본질식별자 (주문번호 + 상품번호) 로 PK 를 잡으면 중복을 막는다
CREATE TABLE ORDER_ITEM2 (ORDER_NO INTEGER, PROD_NO INTEGER, QTY INTEGER, PRIMARY KEY (ORDER_NO, PROD_NO));
INSERT INTO ORDER_ITEM2 VALUES (100, 7, 2);
INSERT INTO ORDER_ITEM2 VALUES (100, 7, 2);
```

실행 결과

```
생성되었습니다.

1개 행이 삽입되었습니다.

오류: UNIQUE constraint failed: ORDER_ITEM2.ORDER_NO, ORDER_ITEM2.PROD_NO
```

TIP 인조식별자를 쓸 때는 본질식별자 칼럼에 **유니크 인덱스(UNIQUE)** 를 따로 만들어 중복을 막는 것이 좋아요.

확인 문제

Q1. 인조식별자의 단점으로 옳은 것은?

① PK 가 복잡해진다
② 중복 데이터가 생길 수 있고 인덱스가 추가로 필요할 수 있다
③ 업무 의미가 너무 강하다
④ 조인이 불가능하다

Q2. 업무에 원래 존재하는 값으로 구성한 식별자는?

① 본질식별자
② 인조식별자
③ 보조식별자
④ 외부식별자

CHAPTER 03

[2과목] SQL 기본

학습 목표

- 관계형 데이터베이스와 SQL 명령의 종류를 안다.
- SELECT 문의 구조와 실행 순서를 안다.
- 문자·숫자·날짜·변환·NULL 관련 함수와 CASE·DECODE 를 쓸 수 있다.
- WHERE 절의 연산자 우선순위와 BETWEEN·IN·LIKE·NULL 조건을 쓸 수 있다.
- GROUP BY, HAVING 으로 그룹별 집계를 하고 ORDER BY 로 정렬한다.
- 등가·비등가 조인과 표준 조인(INNER, OUTER, NATURAL, USING, CROSS)을 쓸 수 있다.

03.1 관계형 데이터베이스 개요

데이터베이스는 여러 사람이 함께 쓰려고 모아 둔 데이터 묶음이고, 이를 관리하는 프로그램을 **DBMS**라고 해요. 그중 데이터를 **표(테이블)** 형태로 저장하고 테이블끼리 관계로 잇는 것이 **관계형 데이터베이스(RDBMS)**입니다. (Oracle, MySQL, SQL Server, PostgreSQL 등)

용어	뜻
테이블 (릴레이션)	행과 열로 된 데이터 저장 단위
행 (Row, 튜플)	데이터 하나 (사원 한 명)
열 (Column, 속성)	데이터의 항목 (이름, 급여)
기본키 (PK)	행을 유일하게 구분하는 칼럼
외래키 (FK)	다른 테이블의 PK 를 참조하는 칼럼

SQL 명령의 종류

종류	명령어	하는 일
DML (조작어)	SELECT, INSERT, UPDATE, DELETE, MERGE	데이터 조회·변경
DDL (정의어)	CREATE, ALTER, DROP, RENAME, TRUNCATE	테이블 등 구조 정의
DCL (제어어)	GRANT, REVOKE	권한 부여·회수
TCL (트랜잭션 제어어)	COMMIT, ROLLBACK, SAVEPOINT	트랜잭션 확정·취소

TIP SELECT 를 따로 **DQL**(질의어)로 나누기도 해요. TRUNCATE 는 데이터를 지우지만 **DDL** 이라는 점이 시험에 자주 나옵니다.

SQL

```
-- 예제 데이터베이스의 세 테이블 맛보기
SELECT * FROM DEPT;
```

실행 결과

```
DEPTNO ¦ DNAME      ¦ LOC
-------+------------+---------
10     ¦ ACCOUNTING ¦ NEW YORK
20     ¦ RESEARCH   ¦ DALLAS
30     ¦ SALES      ¦ CHICAGO
40     ¦ OPERATIONS ¦ BOSTON
(4개 행)
```

확인 문제

Q1. 다음 중 DDL 이 아닌 것은?

① CREATE
② ALTER
③ TRUNCATE
④ GRANT

Q2. COMMIT, ROLLBACK 이 속하는 SQL 종류는?

① DML
② DDL
③ DCL
④ TCL

Q3. 다른 테이블의 기본키를 참조하는 칼럼은?

① 후보키
② 외래키
③ 대체키
④ 슈퍼키

03.2 SELECT 문

```
SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)
SELECT [DISTINCT] 칼럼, 식 [AS 별칭]      -- ⑤ 무엇을 보여 줄까
FROM 테이블                               -- ① 어디서
WHERE 조건                                -- ② 어떤 행을
GROUP BY 칼럼                             -- ③ 어떻게 묶을까
HAVING 그룹 조건                          -- ④ 어떤 그룹을
ORDER BY 칼럼 [ASC¦DESC];                 -- ⑥ 어떤 순서로
```

TIP **실행 순서**: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. 그래서 SELECT 에서 만든 **별칭은 WHERE 에서 쓸 수 없지만 ORDER BY 에서는 쓸 수 있어요.**

```
SQL
-- 칼럼 고르기, 계산식, 별칭
SELECT ENAME AS 이름, SAL AS 월급, SAL * 12 AS 연봉 FROM EMP WHERE DEPTNO = 10;
```

```
실행 결과
이름   ¦ 월급 ¦ 연봉
-------+------+------
CLARK  ¦ 2450 ¦ 29400
KING   ¦ 5000 ¦ 60000
MILLER ¦ 1300 ¦ 15600
(3개 행)
```

```
SQL
-- DISTINCT: 중복 행 제거
SELECT DISTINCT JOB FROM EMP;
```

실행 결과

```
JOB
---------
CLERK
SALESMAN
MANAGER
ANALYST
PRESIDENT
(5개 행)
```

SQL

```
-- 합성 연산자 || 로 문자열 잇기 (SQL Server 는 +)
SELECT ENAME || '의 업무는 ' || JOB || '입니다' AS 소개 FROM EMP WHERE DEPTNO = 10;
```

실행 결과

```
소개
----------------------------
CLARK의 업무는 MANAGER입니다
KING의 업무는 PRESIDENT입니다
MILLER의 업무는 CLERK입니다
(3개 행)
```

주의 별칭에 공백이나 특수문자가 있으면 큰따옴표로 감싸요: `SAL * 12 AS "연 봉"`. 작은따옴표는 문자열 값에만 씁니다.

확인 문제

Q1. SELECT 문의 실행 순서로 옳은 것은?

① SELECT → FROM → WHERE → GROUP BY
② FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
③ FROM → SELECT → WHERE → ORDER BY
④ WHERE → FROM → SELECT → HAVING

Q2. SELECT 절에서 정의한 별칭을 사용할 수 있는 절은?

① WHERE
② GROUP BY
③ HAVING
④ ORDER BY

Q3. 중복된 행을 한 번만 보여 주는 키워드는?

① UNIQUE
② DISTINCT
③ ONLY
④ SINGLE

03.3 함수(Function)

함수는 **단일행 함수**(행마다 결과 하나)와 **다중행 함수**(여러 행에서 결과 하나, 집계 함수)로 나뉘어요. 여기서는 단일행 함수를 봅니다. 오라클은 테이블 없이 계산만 할 때 **DUAL** 테이블을 써요.

문자 함수

함수	결과 예
LOWER / UPPER / INITCAP('sql exPERT')	sql expert / SQL EXPERT / Sql Expert

함수	결과 예
LENGTH('SQLD')	4
SUBSTR('SQL Expert', 5, 3)	Exp (5번째부터 3글자)
INSTR('SQL Expert', 'E')	5 (처음 나오는 위치)
LPAD('7', 3, '0') / RPAD	007
LTRIM / RTRIM / TRIM	왼쪽·오른쪽·양쪽 공백(문자) 제거
REPLACE('ABCB', 'B', 'X')	AXCX
CONCAT('A', 'B')	AB (인수 2개)

SQL
```
SELECT UPPER('sqld') AS U, INITCAP('sql exPERT') AS I, LENGTH('SQLD') AS L,
       SUBSTR('SQL Expert', 5, 3) AS S, INSTR('SQL Expert', 'E') AS P,
       LPAD('7', 3, '0') AS PAD, REPLACE('ABCB', 'B', 'X') AS R
FROM DUAL;
```

실행 결과
```
U    ¦ I          ¦ L ¦ S   ¦ P ¦ PAD ¦ R
-----+------------+---+-----+---+-----+-----
SQLD ¦ Sql Expert ¦ 4 ¦ Exp ¦ 5 ¦ 007 ¦ AXCX
(1개 행)
```

숫자 함수

함수	결과 예
ROUND(123.456, 1) / ROUND(123.456, -1)	123.5 / 120
TRUNC(123.456, 1) / TRUNC(-1.5)	123.4 / -1 (버림)
CEIL(1.1) / FLOOR(1.9)	2 / 1
MOD(7, 3)	1 (나머지)
ABS(-5) / SIGN(-5)	5 / -1

SQL
```
SELECT ROUND(123.456, 1) AS R1, ROUND(123.456, -1) AS R2, TRUNC(123.456, 1) AS T1,
       CEIL(1.1) AS C, FLOOR(-1.1) AS F, MOD(7, 3) AS M, SIGN(-5) AS S
FROM DUAL;
```

실행 결과
```
R1    ¦ R2  ¦ T1    ¦ C ¦ F  ¦ M ¦ S
------+-----+-------+---+----+---+---
123.5 ¦ 120 ¦ 123.4 ¦ 2 ¦ -2 ¦ 1 ¦ -1
(1개 행)
```

날짜 함수 (오라클)

함수	뜻
SYSDATE	현재 날짜·시간
날짜 + 1 / 날짜 - 날짜	하루 뒤 / 두 날짜 사이 일수
ADD_MONTHS(날짜, n)	n 개월 뒤
EXTRACT(YEAR FROM 날짜)	연도만 꺼내기

함수	뜻
TO_CHAR(날짜, 'YYYY-MM-DD')	날짜 → 문자
TO_DATE('2026-11-04', 'YYYY-MM-DD')	문자 → 날짜

SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)

```
SELECT SYSDATE, SYSDATE + 1, ADD_MONTHS(SYSDATE, 3), TO_CHAR(SYSDATE, 'YYYY"년" MM"월"')
FROM DUAL;                          -- 오라클 전용 (실행 시점마다 결과가 다름)
```

TIP **변환**에는 명시적 변환(TO_CHAR, TO_NUMBER, TO_DATE 등 함수 사용)과 암시적 변환(DBMS 가 알아서 바꿈)이 있어요. 암시적 변환은 인덱스를 못 쓰게 만들 수 있어 명시적 변환이 권장됩니다.

NULL 관련 함수

함수	뜻	예 → 결과
NVL(a, b)	a 가 NULL 이면 b	NVL(NULL, 0) → 0
NVL2(a, b, c)	a 가 NULL 이 아니면 b, NULL 이면 c	NVL2(COMM, 'Y', 'N')
NULLIF(a, b)	a = b 이면 NULL, 아니면 a	NULLIF(5, 5) → NULL
COALESCE(a, b, c…)	처음으로 NULL 이 아닌 값	COALESCE(NULL, NULL, 3) → 3

SQL

```
SELECT ENAME, COMM, NVL(COMM, 0) AS NVL, NVL2(COMM, '있음', '없음') AS NVL2,
       NULLIF(SAL, 1250) AS NULLIF, COALESCE(COMM, MGR, 0) AS COAL
FROM EMP WHERE DEPTNO = 30;
```

실행 결과

```
ENAME  ¦ COMM ¦ NVL  ¦ NVL2 ¦ NULLIF ¦ COAL
-------+------+------+------+--------+-----
ALLEN  ¦ 300  ¦ 300  ¦ 있음 ¦ 1600   ¦ 300
WARD   ¦ 500  ¦ 500  ¦ 있음 ¦ NULL   ¦ 500
MARTIN ¦ 1400 ¦ 1400 ¦ 있음 ¦ NULL   ¦ 1400
BLAKE  ¦ NULL ¦ 0    ¦ 없음 ¦ 2850   ¦ 7839
TURNER ¦ 0    ¦ 0    ¦ 있음 ¦ 1500   ¦ 0
JAMES  ¦ NULL ¦ 0    ¦ 없음 ¦ 950    ¦ 7698
(6개 행)
```

CASE 와 DECODE

SQL

```
-- 검색형 CASE: 조건에 따라 다른 값
SELECT ENAME, SAL,
       CASE WHEN SAL >= 3000 THEN '고액'
            WHEN SAL >= 1500 THEN '중간'
            ELSE '일반' END AS 등급,
       DECODE(DEPTNO, 10, '회계', 20, '연구', '기타') AS 부서        -- 오라클 DECODE
FROM EMP WHERE DEPTNO IN (10, 20);
```

```
실행 결과
ENAME  ¦ SAL  ¦ 등급 ¦ 부서
-------+------+------+-----
SMITH  ¦ 800  ¦ 일반 ¦ 연구
JONES  ¦ 2975 ¦ 중간 ¦ 연구
CLARK  ¦ 2450 ¦ 중간 ¦ 회계
SCOTT  ¦ 3000 ¦ 고액 ¦ 연구
KING   ¦ 5000 ¦ 고액 ¦ 회계
ADAMS  ¦ 1100 ¦ 일반 ¦ 연구
FORD   ¦ 3000 ¦ 고액 ¦ 연구
MILLER ¦ 1300 ¦ 일반 ¦ 회계
(8개 행)
```

TIP `DECODE(표현식, 값1, 결과1, 값2, 결과2, 기본값)` 은 `CASE 표현식 WHEN 값1 THEN 결과1 … ELSE 기본값 END` (단순형 CASE) 와 같아요. ELSE 가 없으면 NULL 이 나옵니다.

확인 문제

Q1. SUBSTR('DATABASE', 3, 4) 의 결과는?

① DATA
② TABA
③ ABAS
④ TAB

Q2. ROUND(45.678, -1) 의 결과는?

① 45.7
② 46
③ 50
④ 40

Q3. NVL2(NULL, 'A', 'B') 의 결과는?

① A
② B
③ NULL
④ 오류

Q4. COALESCE(NULL, 10, 20) 의 결과는?

① NULL
② 10
③ 20
④ 30

Q5. NULLIF(100, 100) 의 결과는?

① NULL
② 100
③ 0
④ 200

03.4 WHERE 절

WHERE 절은 **원하는 행만 골라내는 조건**이에요.

연산자	뜻	예
= , <> (!=, ^=) , > , >= , < , <=	비교	SAL >= 3000

연산자	뜻	예
BETWEEN a AND b	a 이상 b 이하	SAL BETWEEN 1000 AND 2000
IN (목록)	목록 중 하나	DEPTNO IN (10, 20)
LIKE '패턴'	패턴 일치 (% 여러 글자, _ 한 글자)	ENAME LIKE 'S%'
IS NULL / IS NOT NULL	NULL 확인	COMM IS NULL
NOT, AND, OR	논리 연산	

TIP **연산자 우선순위**: ( ) → 비교 연산자 → NOT → AND → OR. AND 가 OR 보다 먼저 계산돼요!

SQL
```
SELECT ENAME, SAL FROM EMP WHERE SAL BETWEEN 1250 AND 1600;
```

실행 결과
```
ENAME  ¦ SAL
-------+-----
ALLEN  ¦ 1600
WARD   ¦ 1250
MARTIN ¦ 1250
TURNER ¦ 1500
MILLER ¦ 1300
(5개 행)
```

SQL
```
-- LIKE: 두 번째 글자가 A 인 이름
SELECT ENAME FROM EMP WHERE ENAME LIKE '_A%';
```

실행 결과
```
ENAME
------
WARD
MARTIN
JAMES
(3개 행)
```

SQL
```
-- AND 가 먼저: (DEPTNO = 20 AND SAL > 2000) OR JOB = 'CLERK'
SELECT ENAME, DEPTNO, JOB, SAL FROM EMP
WHERE JOB = 'CLERK' OR DEPTNO = 20 AND SAL > 2000;
```

실행 결과
```
ENAME  ¦ DEPTNO ¦ JOB     ¦ SAL
-------+--------+---------+-----
SMITH  ¦ 20     ¦ CLERK   ¦ 800
JONES  ¦ 20     ¦ MANAGER ¦ 2975
SCOTT  ¦ 20     ¦ ANALYST ¦ 3000
ADAMS  ¦ 20     ¦ CLERK   ¦ 1100
JAMES  ¦ 30     ¦ CLERK   ¦ 950
FORD   ¦ 20     ¦ ANALYST ¦ 3000
MILLER ¦ 10     ¦ CLERK   ¦ 1300
(7개 행)
```

주의 `NOT IN (목록)` 의 목록에 **NULL 이 있으면 결과가 하나도 나오지 않아요.** `DEPTNO NOT IN (10, NULL)` 은 `DEPTNO <> 10 AND DEPTNO <> NULL` 이고, NULL 비교는 UNKNOWN 이기 때문이에요.

```
SQL
SELECT COUNT(*) AS 건수 FROM EMP WHERE DEPTNO NOT IN (10, NULL);
```

```
실행 결과
건수
----
0
(1개 행)
```

확인 문제

Q1. SAL BETWEEN 1000 AND 2000 과 같은 조건은?

① SAL > 1000 AND SAL < 2000
② SAL >= 1000 AND SAL <= 2000
③ SAL >= 1000 OR SAL <= 2000
④ SAL IN (1000, 2000)

Q2. LIKE '_A%' 에 맞는 이름은?

① ADAMS
② WARD
③ ALLEN
④ SMITH

Q3. 연산자 우선순위로 옳은 것은?

① OR → AND → NOT
② NOT → AND → OR
③ AND → OR → NOT
④ 모두 같다

Q4. WHERE DEPTNO NOT IN (10, 20, NULL) 의 결과 행 수는?

① DEPTNO 가 30 인 행 수
② 0 건
③ 전체 행 수
④ 오류

03.5 GROUP BY, HAVING 절

집계 함수(다중행 함수)는 여러 행을 묶어 값 하나를 만들어요: COUNT, SUM, AVG, MAX, MIN, STDDEV, VARIAN. **GROUP BY** 는 묶을 기준, **HAVING** 은 묶은 **그룹에 대한 조건**이에요.

```
SQL
-- 부서별 사원 수, 급여 합계·평균·최대
SELECT DEPTNO, COUNT(*) AS 인원, SUM(SAL) AS 합계, ROUND(AVG(SAL), 1) AS 평균, MAX(SAL) AS 최대
FROM EMP GROUP BY DEPTNO;
```

```
실행 결과
DEPTNO ¦ 인원 ¦ 합계  ¦ 평균   ¦ 최대
-------+------+-------+--------+-----
10     ¦ 3    ¦ 8750  ¦ 2916.7 ¦ 5000
20     ¦ 5    ¦ 10875 ¦ 2175   ¦ 3000
30     ¦ 6    ¦ 9400  ¦ 1566.7 ¦ 2850
(3개 행)
```

```sql
-- HAVING: 평균 급여가 2000 이상인 부서만
SELECT DEPTNO, ROUND(AVG(SAL), 1) AS 평균
FROM EMP GROUP BY DEPTNO HAVING AVG(SAL) >= 2000;
```

```
DEPTNO ¦ 평균
-------+-------
10     ¦ 2916.7
20     ¦ 2175
(2개 행)
```

구분	WHERE	HAVING
대상	개별 행	GROUP BY 로 묶은 그룹
집계 함수 사용	불가	가능
실행 시점	GROUP BY 전	GROUP BY 후

주의 GROUP BY 를 쓰면 SELECT 절에는 **GROUP BY 칼럼과 집계 함수만** 올 수 있어요. `SELECT DEPTNO, ENAME, COUNT(*) FROM EMP GROUP BY DEPTNO` 는 오라클에서 오류입니다. (ENAME 이 그룹마다 여러 개라서)

TIP 집계 함수는 NULL 을 빼고 계산하고, 대상이 하나도 없을 때 COUNT 는 0, 나머지(SUM, AVG...)는 NULL 을 돌려줘요.

```sql
SELECT COUNT(*) AS 건수, SUM(SAL) AS 합계 FROM EMP WHERE DEPTNO = 40;
```

```
건수 ¦ 합계
-----+-----
0    ¦ NULL
(1개 행)
```

확인 문제

Q1. 집계 함수를 조건에 쓸 수 있는 절은?

① WHERE
② HAVING
③ FROM
④ ON

Q2. 조건에 맞는 행이 하나도 없을 때 SUM(SAL) 의 결과는?

① 0
② NULL
③ 오류
④ 1

Q3. GROUP BY DEPTNO 를 쓴 SELECT 절에 올 수 없는 것은?

① DEPTNO
② COUNT(*)
③ MAX(SAL)
④ ENAME

03.6 ORDER BY 절

결과를 **정렬**해요. 기본은 오름차순(ASC), 내림차순은 DESC. 칼럼 이름, **별칭**, **SELECT 절의 순서 번호**로 지정할 수 있어요.

SQL
```
-- 부서는 오름차순, 같은 부서 안에서는 급여 내림차순
SELECT DEPTNO, ENAME, SAL FROM EMP WHERE DEPTNO IN (10, 20) ORDER BY DEPTNO, SAL DESC;
```

실행 결과
```
DEPTNO ¦ ENAME  ¦ SAL
-------+--------+-----
10     ¦ KING   ¦ 5000
10     ¦ CLARK  ¦ 2450
10     ¦ MILLER ¦ 1300
20     ¦ SCOTT  ¦ 3000
20     ¦ FORD   ¦ 3000
20     ¦ JONES  ¦ 2975
20     ¦ ADAMS  ¦ 1100
20     ¦ SMITH  ¦ 800
(8개 행)
```

SQL
```
-- 별칭과 위치 번호로 정렬
SELECT ENAME, SAL * 12 AS 연봉 FROM EMP WHERE DEPTNO = 10 ORDER BY 연봉 DESC;
SELECT ENAME, SAL FROM EMP WHERE DEPTNO = 10 ORDER BY 2;
```

실행 결과
```
ENAME  ¦ 연봉
-------+------
KING   ¦ 60000
CLARK  ¦ 29400
MILLER ¦ 15600
(3개 행)

ENAME  ¦ SAL
-------+-----
MILLER ¦ 1300
CLARK  ¦ 2450
KING   ¦ 5000
(3개 행)
```

TIP **NULL 의 정렬 위치**: 오라클은 NULL 을 **가장 큰 값**으로 봐서 오름차순이면 **맨 뒤**, SQL Server 는 가장 작은 값으로 봐서 맨 앞이에요. 오라클은 `NULLS FIRST / NULLS LAST` 로 바꿀 수 있어요.

SQL
```
-- 이 앱(SQLite)은 NULL 을 작은 값으로 본다 → NULLS LAST 로 오라클처럼
SELECT ENAME, COMM FROM EMP WHERE DEPTNO = 30 ORDER BY COMM NULLS LAST;
```

```
실행 결과
ENAME  ¦ COMM
-------+-----
TURNER ¦ 0
ALLEN  ¦ 300
WARD   ¦ 500
MARTIN ¦ 1400
BLAKE  ¦ NULL
JAMES  ¦ NULL
(6개 행)
```

확인 문제

Q1. ORDER BY 에 대한 설명으로 틀린 것은?

① 기본 정렬은 오름차순이다
② SELECT 절의 별칭을 쓸 수 있다
③ SELECT 절 칼럼의 순서 번호를 쓸 수 있다
④ SQL 문에서 가장 먼저 실행된다

Q2. 오라클에서 COMM 을 오름차순 정렬하면 NULL 의 위치는?

① 맨 앞
② 맨 뒤
③ 가운데
④ 제외된다

03.7 조인

조인은 여러 테이블을 이어서 하나처럼 조회하는 거예요. WHERE 절에 조인 조건을 쓰는 방식(오라클 방식)을 먼저 봅니다.

등가 조인 (EQUI JOIN)

```sql
SELECT E.ENAME, D.DNAME, D.LOC
FROM EMP E, DEPT D
WHERE E.DEPTNO = D.DEPTNO AND E.SAL >= 3000;
```

```
실행 결과
ENAME ¦ DNAME      ¦ LOC
------+------------+---------
SCOTT ¦ RESEARCH   ¦ DALLAS
KING  ¦ ACCOUNTING ¦ NEW YORK
FORD  ¦ RESEARCH   ¦ DALLAS
(3개 행)
```

비등가 조인 (NON-EQUI JOIN)

= 가 아닌 조건(BETWEEN, >, < 등)으로 잇는 조인이에요. 급여로 급여 등급을 찾아봐요.

```sql
SELECT E.ENAME, E.SAL, S.GRADE
FROM EMP E, SALGRADE S
WHERE E.SAL BETWEEN S.LOSAL AND S.HISAL AND E.DEPTNO = 10;
```

실행 결과

```
ENAME  ¦ SAL  ¦ GRADE
-------+------+------
MILLER ¦ 1300 ¦ 2
CLARK  ¦ 2450 ¦ 4
KING   ¦ 5000 ¦ 5
(3개 행)
```

3개 이상 테이블 조인

SQL

```
SELECT E.ENAME, D.DNAME, S.GRADE
FROM EMP E, DEPT D, SALGRADE S
WHERE E.DEPTNO = D.DEPTNO
  AND E.SAL BETWEEN S.LOSAL AND S.HISAL
  AND E.JOB = 'MANAGER';
```

실행 결과

```
ENAME ¦ DNAME      ¦ GRADE
------+------------+------
BLAKE ¦ SALES      ¦ 4
CLARK ¦ ACCOUNTING ¦ 4
JONES ¦ RESEARCH   ¦ 4
(3개 행)
```

TIP 테이블 N 개를 조인하려면 조인 조건이 **최소 N-1 개** 필요해요. 조건을 빠뜨리면 모든 조합이 나오는 **카티션 곱**이 됩니다. (EMP 14건 × DEPT 4건 = 56건)

SQL

```
SELECT COUNT(*) AS 카티션곱 FROM EMP, DEPT;
```

실행 결과

```
카티션곱
--------
56
(1개 행)
```

오라클 외부 조인 (+)

조인 조건에 맞지 않는 행도 보여 주려면, **데이터가 없는 쪽**에 (+) 를 붙여요. (오라클 전용)

SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)

```
SELECT D.DEPTNO, D.DNAME, E.ENAME
FROM DEPT D, EMP E
WHERE D.DEPTNO = E.DEPTNO(+);      -- 사원이 없는 40번 부서도 나온다
```

확인 문제

Q1. 급여 범위로 등급을 찾는 것처럼 = 이 아닌 조건으로 잇는 조인은?

① 등가 조인
② 비등가 조인
③ 셀프 조인
④ 자연 조인

Q2. 테이블 4개를 조인할 때 필요한 최소 조인 조건 수는?

① 2
② 3
③ 4
④ 6

Q3. 조인 조건 없이 EMP(14건)와 DEPT(4건)를 조회하면 결과 건수는?

① 14
② 18
③ 56
④ 4

03.8 표준 조인

ANSI 표준 조인은 조인 조건을 **FROM 절**에 써요. WHERE 에는 일반 조건만 남아 읽기 쉬워집니다.

조인	형식	특징
INNER JOIN	A INNER JOIN B ON 조건	조건에 맞는 행만 (INNER 생략 가능)
NATURAL JOIN	A NATURAL JOIN B	**이름이 같은 모든 칼럼**으로 등가 조인, 별칭·ON 사용 불가
USING	A JOIN B USING (칼럼)	지정한 같은 이름 칼럼으로 조인, 그 칼럼에 별칭 불가
LEFT OUTER JOIN	A LEFT JOIN B ON 조건	왼쪽 A 는 모두 + 오른쪽은 맞는 것만 (없으면 NULL)
RIGHT OUTER JOIN	A RIGHT JOIN B ON 조건	오른쪽 B 는 모두
FULL OUTER JOIN	A FULL JOIN B ON 조건	양쪽 모두
CROSS JOIN	A CROSS JOIN B	카티션 곱

SQL
```
SELECT E.ENAME, D.DNAME FROM EMP E INNER JOIN DEPT D ON E.DEPTNO = D.DEPTNO WHERE D.DEPTNO = 10;
```

실행 결과
```
ENAME  ¦ DNAME
-------+-----------
CLARK  ¦ ACCOUNTING
KING   ¦ ACCOUNTING
MILLER ¦ ACCOUNTING
(3개 행)
```

SQL
```
-- USING: 조인 칼럼 DEPTNO 에는 별칭을 붙이지 않는다
SELECT DEPTNO, E.ENAME, D.LOC FROM EMP E JOIN DEPT D USING (DEPTNO) WHERE DEPTNO = 10;
```

실행 결과
```
DEPTNO ¦ ENAME  ¦ LOC
-------+--------+---------
10     ¦ CLARK  ¦ NEW YORK
10     ¦ KING   ¦ NEW YORK
10     ¦ MILLER ¦ NEW YORK
(3개 행)
```

SQL
```
-- LEFT OUTER JOIN: 사원이 없는 40번 부서도 나온다
SELECT D.DEPTNO, D.DNAME, COUNT(E.EMPNO) AS 인원
FROM DEPT D LEFT OUTER JOIN EMP E ON D.DEPTNO = E.DEPTNO
GROUP BY D.DEPTNO, D.DNAME ORDER BY D.DEPTNO;
```

실행 결과

```
DEPTNO ¦ DNAME      ¦ 인원
-------+------------+-----
10     ¦ ACCOUNTING ¦ 3
20     ¦ RESEARCH   ¦ 5
30     ¦ SALES      ¦ 6
40     ¦ OPERATIONS ¦ 0
(4개 행)
```

SQL

```
-- FULL OUTER JOIN: 양쪽에서 짝이 없는 행도 모두 (부서 없는 사원을 하나 추가해서 확인)
INSERT INTO EMP (EMPNO, ENAME, SAL) VALUES (9999, 'NEWBIE', 1000);
SELECT E.ENAME, D.DNAME FROM EMP E FULL OUTER JOIN DEPT D ON E.DEPTNO = D.DEPTNO
WHERE E.DEPTNO IS NULL OR D.DEPTNO = 40 OR E.EMPNO IS NULL;
```

실행 결과

```
1개 행이 삽입되었습니다.

ENAME  ¦ DNAME
-------+----------
NEWBIE ¦ NULL
NULL   ¦ OPERATIONS
(2개 행)
```

TIP LEFT JOIN 에서 `COUNT(*)` 를 쓰면 40번 부서도 1 로 세어져요(NULL 로 채운 행 하나). 사원 수를 정확히 세려면 `COUNT(E.EMPNO)` 처럼 **오른쪽 테이블의 칼럼**을 세야 합니다.

확인 문제

Q1. 이름이 같은 모든 칼럼으로 자동으로 등가 조인하는 것은?

① INNER JOIN
② NATURAL JOIN
③ CROSS JOIN
④ OUTER JOIN

Q2. USING (DEPTNO) 조인에서 틀린 SELECT 절은?

① SELECT DEPTNO
② SELECT E.DEPTNO
③ SELECT E.ENAME
④ SELECT D.DNAME

Q3. DEPT 를 기준으로 사원이 없는 부서도 모두 보이려면? (FROM DEPT D ___ EMP E ON ...)

① LEFT OUTER JOIN
② RIGHT OUTER JOIN
③ INNER JOIN
④ NATURAL JOIN

Q4. DEPT LEFT JOIN EMP 후 부서별 COUNT(*) 를 하면 사원이 없는 40번 부서의 값은?

① 0
② 1
③ NULL
④ 결과에 없다

CHAPTER 04

[2과목] SQL 활용

학습 목표

- 서브쿼리의 종류(위치·반환 행 수·연관 여부)를 구분해 쓸 수 있다.
- UNION, UNION ALL, INTERSECT, MINUS(EXCEPT)의 차이를 안다.
- ROLLUP, CUBE, GROUPING SETS 의 결과 모양을 읽을 수 있다.
- 순위·집계·행 순서·비율 윈도우 함수를 쓸 수 있다.
- TOP-N 쿼리(ROWNUM, ROW_NUMBER, FETCH FIRST)를 안다.
- 셀프 조인과 계층형 질의(CONNECT BY)를 읽을 수 있다.
- PIVOT/UNPIVOT 과 정규 표현식 함수를 안다.

04.1 서브쿼리

서브쿼리는 SQL 문 안에 들어 있는 또 하나의 SELECT 문이에요. 괄호로 감싸요.

기준	종류
위치	스칼라 서브쿼리 (SELECT 절, 값 하나), 인라인 뷰 (FROM 절, 테이블처럼), 중첩 서브쿼리 (WHERE·HAVING 절)
반환 행 수	단일행 (=, >, < 등 사용), 다중행 (IN, ANY, ALL, EXISTS 사용), 다중 칼럼
메인쿼리와의 관계	비연관 서브쿼리 (혼자 실행 가능), 연관(상관) 서브쿼리 (메인쿼리 칼럼을 사용)

SQL

```
-- 단일행 서브쿼리: 평균 급여보다 많이 받는 사원
SELECT ENAME, SAL FROM EMP WHERE SAL > (SELECT AVG(SAL) FROM EMP);
```

실행 결과

```
ENAME ¦ SAL
------+-----
JONES ¦ 2975
BLAKE ¦ 2850
CLARK ¦ 2450
SCOTT ¦ 3000
KING  ¦ 5000
FORD  ¦ 3000
(6개 행)
```

SQL

```
-- 다중행 서브쿼리: 부서별 최고 급여를 받는 사원 (IN, 다중 칼럼)
SELECT DEPTNO, ENAME, SAL FROM EMP
WHERE (DEPTNO, SAL) IN (SELECT DEPTNO, MAX(SAL) FROM EMP GROUP BY DEPTNO);
```

실행 결과

```
DEPTNO ¦ ENAME ¦ SAL
-------+-------+-----
30     ¦ BLAKE ¦ 2850
20     ¦ SCOTT ¦ 3000
10     ¦ KING  ¦ 5000
20     ¦ FORD  ¦ 3000
(4개 행)
```

다중행 연산자	뜻
IN	결과 중 하나와 같으면
> ANY (SOME)	결과 중 **하나라도** 보다 크면 = 최솟값보다 크면
> ALL	결과 **모두**보다 크면 = 최댓값보다 크면
EXISTS	결과가 **한 건이라도 있으면** 참

```sql
-- 연관 서브쿼리 + EXISTS: 사원이 한 명이라도 있는 부서
SELECT D.DEPTNO, D.DNAME FROM DEPT D
WHERE EXISTS (SELECT 1 FROM EMP E WHERE E.DEPTNO = D.DEPTNO);
```

실행 결과

```
DEPTNO ¦ DNAME
-------+----------
10     ¦ ACCOUNTING
20     ¦ RESEARCH
30     ¦ SALES
(3개 행)
```

```sql
-- 연관 서브쿼리: 자기 부서 평균보다 많이 받는 사원
SELECT E.DEPTNO, E.ENAME, E.SAL FROM EMP E
WHERE E.SAL > (SELECT AVG(SAL) FROM EMP WHERE DEPTNO = E.DEPTNO);
```

실행 결과

```
DEPTNO ¦ ENAME ¦ SAL
-------+-------+-----
30     ¦ ALLEN ¦ 1600
20     ¦ JONES ¦ 2975
30     ¦ BLAKE ¦ 2850
20     ¦ SCOTT ¦ 3000
10     ¦ KING  ¦ 5000
20     ¦ FORD  ¦ 3000
(6개 행)
```

```sql
-- 스칼라 서브쿼리(SELECT 절)와 인라인 뷰(FROM 절)
SELECT E.ENAME, (SELECT DNAME FROM DEPT D WHERE D.DEPTNO = E.DEPTNO) AS 부서명
FROM (SELECT * FROM EMP WHERE SAL >= 3000) E;
```

실행 결과

```
ENAME ¦ 부서명
------+----------
SCOTT ¦ RESEARCH
KING  ¦ ACCOUNTING
FORD  ¦ RESEARCH
(3개 행)
```

주의 단일행 연산자(=, > 등) 뒤의 서브쿼리가 **여러 행**을 돌려주면 오류가 나요. 스칼라 서브쿼리도 한 행·한 칼럼만 돌려줘야 합니다.

TIP **뷰(View)**는 SELECT 문을 이름 붙여 저장한 가상 테이블이에요. 장점: 독립성(테이블이 바뀌어도 응용 프로그램 영향 적음), 편리성(복잡한 질의를 단순하게), 보안성(필요한 칼럼만 보여 줌).

확인 문제

Q1. FROM 절에 쓰여 테이블처럼 사용되는 서브쿼리는?

① 스칼라 서브쿼리
② 인라인 뷰
③ 중첩 서브쿼리
④ 연관 서브쿼리

Q2. SAL > ALL (1000, 2000, 3000) 과 같은 조건은?

① SAL > 1000
② SAL > 2000
③ SAL > 3000
④ SAL > 6000

Q3. SAL > ANY (1000, 2000, 3000) 과 같은 조건은?

① SAL > 1000
② SAL > 2000
③ SAL > 3000
④ SAL IN (1000, 2000, 3000)

Q4. 메인쿼리의 칼럼을 서브쿼리 안에서 사용하는 것은?

① 비연관 서브쿼리
② 연관 서브쿼리
③ 스칼라 서브쿼리
④ 인라인 뷰

04.2 집합 연산자

두 SELECT 결과를 **위아래로** 합치거나 비교해요. 두 쿼리의 **칼럼 수와 자료형이 같아야** 합니다.

연산자	결과	중복	정렬
UNION	합집합	**제거**	정렬될 수 있음
UNION ALL	합집합	**그대로**	정렬 안 함 (가장 빠름)
INTERSECT	교집합	제거	
MINUS (SQL Server·SQLite: EXCEPT)	차집합	제거	

```sql
-- UNION 과 UNION ALL 비교: 10번 부서 업무 + 20번 부서 업무
SELECT JOB FROM EMP WHERE DEPTNO = 10
UNION
SELECT JOB FROM EMP WHERE DEPTNO = 20;
SELECT COUNT(*) AS UNION_ALL_건수 FROM (SELECT JOB FROM EMP WHERE DEPTNO = 10 UNION ALL SELECT JOB FROM EMP WHERE DEPTNO = 20);
```

실행 결과

```
JOB
---------
ANALYST
CLERK
MANAGER
PRESIDENT
(4개 행)

UNION_ALL_건수
--------------
8
(1개 행)
```

SQL

```sql
-- INTERSECT: 10번과 20번 부서에 모두 있는 업무 / EXCEPT(MINUS): 20번에만 있는 업무
SELECT JOB FROM EMP WHERE DEPTNO = 10 INTERSECT SELECT JOB FROM EMP WHERE DEPTNO = 20;
SELECT JOB FROM EMP WHERE DEPTNO = 20 EXCEPT SELECT JOB FROM EMP WHERE DEPTNO = 10;
```

실행 결과

```
JOB
-------
CLERK
MANAGER
(2개 행)

JOB
-------
ANALYST
(1개 행)
```

TIP 결과의 칼럼 이름은 **첫 번째 SELECT** 를 따르고, ORDER BY 는 **맨 마지막에 한 번만** 쓸 수 있어요.

확인 문제

Q1. 중복을 제거하지 않아 가장 빠른 집합 연산자는?

① UNION
② UNION ALL
③ INTERSECT
④ MINUS

Q2. 오라클의 MINUS 와 같은 SQL 표준(SQL Server) 연산자는?

① UNION
② INTERSECT
③ EXCEPT
④ DIFFERENCE

Q3. 집합 연산자 결과의 칼럼 이름은?

① 첫 번째 SELECT 의 칼럼 이름
② 마지막 SELECT 의 칼럼 이름
③ 자동으로 COL1, COL2
④ 두 이름을 합친 이름

04.3 그룹 함수

GROUP BY 로 그룹별 집계를 할 때, **소계·합계 행을 함께** 만들어 주는 함수예요. (오라클·SQL Server 지원, 이 앱의 SQLite 는 미지원이라 같은 결과를 UNION ALL 로 보여 줘요)

함수	만드는 집계	GROUP BY ROLLUP(A, B) 예
ROLLUP(A, B)	(A, B) + (A 소계) + (전체 합계)	**순서가 중요**, N 개 칼럼이면 N+1 단계
CUBE(A, B)	가능한 **모든 조합**: (A,B), (A), (B), 전체	2^N 가지
GROUPING SETS(A, B)	**지정한 것만** 각각: (A), (B)	순서 무관, 전체 합계 없음

SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)
```
-- 오라클: 부서별·업무별 급여 합계 + 부서 소계 + 전체 합계
SELECT DEPTNO, JOB, SUM(SAL) AS 합계
FROM EMP WHERE DEPTNO IN (10, 20)
GROUP BY ROLLUP(DEPTNO, JOB);
```

SQL
```
-- 위 ROLLUP 과 같은 결과 (소계 행의 JOB, 합계 행의 DEPTNO 는 NULL)
SELECT DEPTNO, JOB, SUM(SAL) AS 합계 FROM EMP WHERE DEPTNO IN (10, 20) GROUP BY DEPTNO, JOB
UNION ALL SELECT DEPTNO, NULL, SUM(SAL) FROM EMP WHERE DEPTNO IN (10, 20) GROUP BY DEPTNO
UNION ALL SELECT NULL, NULL, SUM(SAL) FROM EMP WHERE DEPTNO IN (10, 20)
ORDER BY 1 NULLS LAST, 2 NULLS LAST;
```

실행 결과
```
DEPTNO ¦ JOB       ¦ 합계
-------+-----------+------
10     ¦ CLERK     ¦ 1300
10     ¦ MANAGER   ¦ 2450
10     ¦ PRESIDENT ¦ 5000
10     ¦ NULL      ¦ 8750
20     ¦ ANALYST   ¦ 6000
20     ¦ CLERK     ¦ 1900
20     ¦ MANAGER   ¦ 2975
20     ¦ NULL      ¦ 10875
NULL   ¦ NULL      ¦ 19625
(9개 행)
```

GROUPING 함수

소계·합계 때문에 생긴 NULL 이면 **1**, 원래 데이터면 **0** 을 돌려줘요. CASE 와 함께 '소계', '합계' 글자를 붙일 때 씁니다.

SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)
```
SELECT CASE GROUPING(DEPTNO) WHEN 1 THEN '전체 합계' ELSE TO_CHAR(DEPTNO) END AS 부서,
       CASE GROUPING(JOB) WHEN 1 THEN '소계' ELSE JOB END AS 업무,
       SUM(SAL) AS 합계
FROM EMP GROUP BY ROLLUP(DEPTNO, JOB);
```

SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)
```
-- CUBE: (부서, 업무) + 부서별 + 업무별 + 전체 / GROUPING SETS: 부서별 + 업무별만
SELECT DEPTNO, JOB, SUM(SAL) FROM EMP GROUP BY CUBE(DEPTNO, JOB);
SELECT DEPTNO, JOB, SUM(SAL) FROM EMP GROUP BY GROUPING SETS(DEPTNO, JOB);
```

TIP **행 수 계산**: 10·20번 부서의 (부서, 업무) 조합은 6개예요. ROLLUP(DEPTNO, JOB) = 6 + 부서 소계 2 + 합계 1 = **9행**, GROUPING SETS(DEPTNO, JOB) = 부서 2 + 업무 4 = **6행**.

확인 문제

Q1. GROUP BY ROLLUP(A, B) 가 만들지 않는 집계는?

① (A, B) 별 집계
② A 별 소계
③ 전체 합계
④ B 별 소계

Q2. 지정한 칼럼별 집계만 각각 만들고 전체 합계는 만들지 않는 것은?

① ROLLUP
② CUBE
③ GROUPING SETS
④ GROUPING

Q3. 칼럼 3개로 CUBE 를 하면 만들어지는 집계 조합 수는?

① 3
② 4
③ 8
④ 6

Q4. GROUPING(칼럼) 이 1 을 돌려주는 경우는?

① 칼럼 값이 1 일 때
② 소계·합계 때문에 생긴 NULL 일 때
③ 원래 데이터가 NULL 일 때
④ 그룹이 하나뿐일 때

04.4 윈도우 함수

윈도우 함수는 행을 줄이지 않고, 각 행 옆에 **순위·누적합·앞뒤 행 값** 같은 계산 결과를 붙여 줘요.

SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)
```
함수() OVER ([PARTITION BY 칼럼] [ORDER BY 칼럼] [ROWS¦RANGE BETWEEN 시작 AND 끝])
```

순위 함수

함수	같은 값 처리	예 (3000, 3000, 2975)
RANK	같은 순위, 다음 순위는 **건너뜀**	1, 1, 3
DENSE_RANK	같은 순위, 다음 순위 **이어짐**	1, 1, 2
ROW_NUMBER	같은 값도 **다른 순위**	1, 2, 3

SQL
```
SELECT ENAME, SAL,
       RANK() OVER (ORDER BY SAL DESC) AS RNK,
       DENSE_RANK() OVER (ORDER BY SAL DESC) AS DRNK,
       ROW_NUMBER() OVER (ORDER BY SAL DESC, ENAME) AS RN
FROM EMP WHERE DEPTNO = 20;
```

실행 결과
```
ENAME ¦ SAL  ¦ RNK ¦ DRNK ¦ RN
------+------+-----+------+---
FORD  ¦ 3000 ¦ 1   ¦ 1    ¦ 1
SCOTT ¦ 3000 ¦ 1   ¦ 1    ¦ 2
JONES ¦ 2975 ¦ 3   ¦ 2    ¦ 3
ADAMS ¦ 1100 ¦ 4   ¦ 3    ¦ 4
SMITH ¦ 800  ¦ 5   ¦ 4    ¦ 5
(5개 행)
```

집계 함수 + PARTITION BY, 누적 합계

```sql
SELECT DEPTNO, ENAME, SAL,
       SUM(SAL) OVER (PARTITION BY DEPTNO) AS 부서합계,
       SUM(SAL) OVER (PARTITION BY DEPTNO ORDER BY SAL, ENAME
                      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS 누적합
FROM EMP WHERE DEPTNO IN (10, 30);
```

실행 결과

```
DEPTNO ¦ ENAME  ¦ SAL  ¦ 부서합계 ¦ 누적합
-------+--------+------+----------+-------
10     ¦ MILLER ¦ 1300 ¦ 8750     ¦ 1300
10     ¦ CLARK  ¦ 2450 ¦ 8750     ¦ 3750
10     ¦ KING   ¦ 5000 ¦ 8750     ¦ 8750
30     ¦ JAMES  ¦ 950  ¦ 9400     ¦ 950
30     ¦ MARTIN ¦ 1250 ¦ 9400     ¦ 2200
30     ¦ WARD   ¦ 1250 ¦ 9400     ¦ 3450
30     ¦ TURNER ¦ 1500 ¦ 9400     ¦ 4950
30     ¦ ALLEN  ¦ 1600 ¦ 9400     ¦ 6550
30     ¦ BLAKE  ¦ 2850 ¦ 9400     ¦ 9400
(9개 행)
```

TIP **ROWS** 는 물리적인 행 개수로, **RANGE** 는 ORDER BY 값의 범위로 창(window)을 정해요. ORDER BY 만 쓰고 범위를 생략하면 `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` 라서 **같은 값을 가진 행은 함께 더해져요.**

```sql
-- 범위를 생략하면 RANGE: 급여 1250 인 두 사람은 누적합이 같다
SELECT ENAME, SAL, SUM(SAL) OVER (ORDER BY SAL) AS RANGE_누적 FROM EMP WHERE DEPTNO = 30;
```

실행 결과

```
ENAME  ¦ SAL  ¦ RANGE_누적
-------+------+-----------
JAMES  ¦ 950  ¦ 950
WARD   ¦ 1250 ¦ 3450
MARTIN ¦ 1250 ¦ 3450
TURNER ¦ 1500 ¦ 4950
ALLEN  ¦ 1600 ¦ 6550
BLAKE  ¦ 2850 ¦ 9400
(6개 행)
```

행 순서 함수

함수	뜻
FIRST_VALUE / LAST_VALUE	창의 첫 / 마지막 행 값
LAG(칼럼, n) / LEAD(칼럼, n)	n 행 **앞** / **뒤** 의 값 (기본 1)

```sql
SELECT ENAME, HIREDATE, SAL,
       LAG(SAL) OVER (ORDER BY HIREDATE) AS 이전입사자급여,
       LEAD(ENAME) OVER (ORDER BY HIREDATE) AS 다음입사자,
       FIRST_VALUE(ENAME) OVER (ORDER BY HIREDATE) AS 첫입사자
FROM EMP WHERE DEPTNO = 10;
```

실행 결과

```
ENAME  ¦ HIREDATE   ¦ SAL  ¦ 이전입사자급여 ¦ 다음입사자 ¦ 첫입사자
-------+------------+------+---------------+------------+---------
CLARK  ¦ 1981-06-09 ¦ 2450 ¦ NULL          ¦ KING       ¦ CLARK
KING   ¦ 1981-11-17 ¦ 5000 ¦ 2450          ¦ MILLER     ¦ CLARK
MILLER ¦ 1982-01-23 ¦ 1300 ¦ 5000          ¦ NULL       ¦ CLARK
(3개 행)
```

비율 함수

함수	뜻
RATIO_TO_REPORT(칼럼)	전체 합에서 차지하는 비율 (오라클)
PERCENT_RANK()	(순위 - 1) / (전체 행 수 - 1), 0 ~ 1
CUME_DIST()	현재 행 이하 값의 누적 비율
NTILE(n)	전체를 n 개 그룹으로 나눈 번호

SQL

```
SELECT ENAME, SAL,
       ROUND(PERCENT_RANK() OVER (ORDER BY SAL), 2) AS PR,
       ROUND(CUME_DIST() OVER (ORDER BY SAL), 2) AS CD,
       NTILE(2) OVER (ORDER BY SAL) AS 그룹,
       ROUND(SAL * 1.0 / SUM(SAL) OVER (), 3) AS 비율          -- RATIO_TO_REPORT(SAL) OVER () 와 같음
FROM EMP WHERE DEPTNO = 20;
```

실행 결과

```
ENAME ¦ SAL  ¦ PR   ¦ CD  ¦ 그룹 ¦ 비율
------+------+------+-----+------+------
SMITH ¦ 800  ¦ 0    ¦ 0.2 ¦ 1    ¦ 0.074
ADAMS ¦ 1100 ¦ 0.25 ¦ 0.4 ¦ 1    ¦ 0.101
JONES ¦ 2975 ¦ 0.5  ¦ 0.6 ¦ 1    ¦ 0.274
SCOTT ¦ 3000 ¦ 0.75 ¦ 1   ¦ 2    ¦ 0.276
FORD  ¦ 3000 ¦ 0.75 ¦ 1   ¦ 2    ¦ 0.276
(5개 행)
```

확인 문제

Q1. 급여 (500, 400, 400, 300) 에 DENSE_RANK() OVER (ORDER BY SAL DESC) 를 하면 300 의 순위는?

① 4
② 3
③ 2
④ 1

Q2. 같은 급여 (500, 400, 400, 300) 에 RANK 를 하면 300 의 순위는?

① 4
② 3
③ 2
④ 1

Q3. 이전 행의 값을 가져오는 윈도우 함수는?

① LAG
② LEAD
③ FIRST_VALUE
④ NTILE

Q4. 윈도우 범위를 생략하고 ORDER BY 만 쓰면 기본 범위는?

① ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
② RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
③ ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
④ 현재 행만

Q5. 같은 값이 있어도 반드시 서로 다른 순위를 매기는 함수는?

① RANK
② DENSE_RANK
③ ROW_NUMBER
④ CUME_DIST

04.5 TOP-N 쿼리

"급여 상위 3명"처럼 **위에서 N 개만** 꺼내는 쿼리예요. DBMS 마다 방법이 달라요.

방법	DBMS	예
ROWNUM	오라클	WHERE ROWNUM <= 3
TOP (N)	SQL Server	SELECT TOP(3) ...
FETCH FIRST N ROWS ONLY	표준 (오라클 12c+)	ORDER BY ... FETCH FIRST 3 ROWS ONLY
LIMIT N	MySQL, PostgreSQL, SQLite	ORDER BY ... LIMIT 3
ROW_NUMBER()	윈도우 함수 (공통)	인라인 뷰에서 순번 후 WHERE RN <= 3

주의 오라클 ROWNUM 은 **ORDER BY 보다 먼저** 매겨져요. 그래서 정렬된 상위 N 개를 얻으려면 **인라인 뷰에서 먼저 정렬**한 뒤 바깥에서 ROWNUM 을 걸어야 합니다. 또 `ROWNUM = 2` 나 `ROWNUM > 1` 은 결과가 나오지 않아요(1번이 없으면 2번도 생기지 않음).

SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)
```
-- 오라클: 잘못된 예 (아무 3명을 뽑은 뒤 정렬)
SELECT ENAME, SAL FROM EMP WHERE ROWNUM <= 3 ORDER BY SAL DESC;
-- 오라클: 올바른 예 (정렬 먼저, 그다음 3명)
SELECT * FROM (SELECT ENAME, SAL FROM EMP ORDER BY SAL DESC) WHERE ROWNUM <= 3;
```

SQL
```
-- 공통 방법: ROW_NUMBER 로 순번을 매기고 바깥에서 거르기
SELECT ENAME, SAL FROM (
  SELECT ENAME, SAL, ROW_NUMBER() OVER (ORDER BY SAL DESC, ENAME) AS RN FROM EMP
) WHERE RN <= 3;
```

실행 결과
```
ENAME ¦ SAL
------+-----
KING  ¦ 5000
FORD  ¦ 3000
SCOTT ¦ 3000
(3개 행)
```

SQL
```
-- 공동 순위까지 포함하려면 RANK (FETCH FIRST 3 ROWS WITH TIES 와 같은 효과)
SELECT ENAME, SAL FROM (SELECT ENAME, SAL, RANK() OVER (ORDER BY SAL DESC) AS R FROM EMP) WHERE R <= 2;
```

```
실행 결과
ENAME ¦ SAL
------+-----
KING  ¦ 5000
SCOTT ¦ 3000
FORD  ¦ 3000
(3개 행)
```

확인 문제

Q1. 오라클에서 ROWNUM 에 대한 설명으로 옳은 것은?

① ORDER BY 다음에 매겨진다
② WHERE ROWNUM = 2 는 결과가 나오지 않는다
③ SQL Server 에서도 쓸 수 있다
④ 항상 정렬된 상위 N 개를 준다

Q2. SQL Server 에서 상위 3건을 조회할 때 쓰는 것은?

① LIMIT 3
② TOP(3)
③ ROWNUM <= 3
④ FIRST 3

Q3. 정렬된 상위 N 건을 오라클 ROWNUM 으로 바르게 구하는 방법은?

① WHERE ROWNUM <= N ORDER BY ...
② 인라인 뷰에서 ORDER BY 한 뒤 바깥에서 ROWNUM <= N
③ ORDER BY ROWNUM
④ GROUP BY ROWNUM

04.6 계층형 질의와 셀프 조인

셀프 조인

같은 테이블을 **별칭을 다르게** 붙여 두 번 쓰는 조인이에요. 사원과 관리자처럼 자기 참조 관계에 씁니다.

```sql
SELECT W.EMPNO, W.ENAME AS 사원, M.ENAME AS 관리자
FROM EMP W LEFT OUTER JOIN EMP M ON W.MGR = M.EMPNO
WHERE W.DEPTNO = 10;
```

```
실행 결과
EMPNO ¦ 사원   ¦ 관리자
------+--------+-------
7782  ¦ CLARK  ¦ KING
7839  ¦ KING   ¦ NULL
7934  ¦ MILLER ¦ CLARK
(3개 행)
```

계층형 질의 (오라클 CONNECT BY)

조직도처럼 **위아래 관계(트리)** 를 따라 내려가며 조회해요.

구문	뜻
START WITH 조건	시작(루트) 행
CONNECT BY PRIOR 자식 = 부모	**PRIOR 쪽이 부모 → 순방향** (위에서 아래로)
LEVEL	깊이 (루트 = 1)

구문	뜻
CONNECT_BY_ISLEAF	자식이 없으면 1
SYS_CONNECT_BY_PATH(칼럼, '/')	루트부터의 경로
ORDER SIBLINGS BY	같은 레벨(형제)끼리 정렬

SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)
```
SELECT LEVEL, LPAD(' ', 2 * (LEVEL - 1)) ¦¦ ENAME AS 조직도, EMPNO, MGR
FROM EMP
START WITH MGR IS NULL                -- KING 부터
CONNECT BY PRIOR EMPNO = MGR          -- 부모(PRIOR)의 EMPNO = 자식의 MGR → 아래로
ORDER SIBLINGS BY ENAME;
```

TIP `CONNECT BY PRIOR MGR = EMPNO` 처럼 바꾸면 **역방향**(자식 → 부모, 아래에서 위로)이에요. "PRIOR 가 붙은 칼럼이 이전 행"이라고 기억하세요.

SQL
```
-- 같은 트리를 표준 SQL 재귀 CTE 로 (WITH RECURSIVE: 이 앱에서 실행 가능)
WITH RECURSIVE T (LVL, ENAME, EMPNO, PATH) AS (
  SELECT 1, ENAME, EMPNO, ENAME FROM EMP WHERE MGR IS NULL
  UNION ALL
  SELECT T.LVL + 1, E.ENAME, E.EMPNO, T.PATH ¦¦ '/' ¦¦ E.ENAME
  FROM EMP E JOIN T ON E.MGR = T.EMPNO
)
SELECT LVL AS 레벨, LPAD(' ', 2 * (LVL - 1)) ¦¦ ENAME AS 조직도, PATH AS 경로 FROM T ORDER BY PATH;
```

실행 결과
```
레벨 ¦ 조직도      ¦ 경로
-----+-------------+----------------------
1    ¦ KING        ¦ KING
2    ¦   BLAKE     ¦ KING/BLAKE
3    ¦     ALLEN   ¦ KING/BLAKE/ALLEN
3    ¦     JAMES   ¦ KING/BLAKE/JAMES
3    ¦     MARTIN  ¦ KING/BLAKE/MARTIN
3    ¦     TURNER  ¦ KING/BLAKE/TURNER
3    ¦     WARD    ¦ KING/BLAKE/WARD
2    ¦   CLARK     ¦ KING/CLARK
3    ¦     MILLER  ¦ KING/CLARK/MILLER
2    ¦   JONES     ¦ KING/JONES
3    ¦     FORD    ¦ KING/JONES/FORD
4    ¦       SMITH ¦ KING/JONES/FORD/SMITH
3    ¦     SCOTT   ¦ KING/JONES/SCOTT
4    ¦       ADAMS ¦ KING/JONES/SCOTT/ADAMS
(14개 행)
```

확인 문제

Q1. CONNECT BY PRIOR EMPNO = MGR 의 전개 방향은?

① 부모에서 자식으로 (순방향)
② 자식에서 부모로 (역방향)
③ 형제끼리
④ 방향 없음

Q2. 계층형 질의에서 루트 행의 LEVEL 값은?

① 0
② 1
③ NULL
④ -1

Q3. 같은 레벨의 형제 행끼리 정렬하는 구문은?

① ORDER BY LEVEL
② ORDER SIBLINGS BY
③ GROUP BY LEVEL
④ CONNECT BY ORDER

Q4. 자식이 없는 행(잎)이면 1 을 돌려주는 것은?

① LEVEL
② CONNECT_BY_ISLEAF
③ CONNECT_BY_ROOT
④ PRIOR

04.7 PIVOT 절과 UNPIVOT 절

- **PIVOT**: 행 값을 **열로** 돌려 세워요. (업무별 행 → 부서를 열로)
- **UNPIVOT**: 열을 **행으로** 내려요.

SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)
```
-- 오라클 PIVOT: 업무별로 부서 10·20·30 의 급여 합계를 열로
SELECT * FROM (SELECT JOB, DEPTNO, SAL FROM EMP)
PIVOT (SUM(SAL) FOR DEPTNO IN (10 AS D10, 20 AS D20, 30 AS D30));
```

SQL
```
-- 같은 결과를 CASE 로 (모든 DBMS 에서 가능)
SELECT JOB,
       SUM(CASE WHEN DEPTNO = 10 THEN SAL END) AS D10,
       SUM(CASE WHEN DEPTNO = 20 THEN SAL END) AS D20,
       SUM(CASE WHEN DEPTNO = 30 THEN SAL END) AS D30
FROM EMP GROUP BY JOB ORDER BY JOB;
```

실행 결과
```
JOB       ¦ D10  ¦ D20  ¦ D30
----------+------+------+-----
ANALYST   ¦ NULL ¦ 6000 ¦ NULL
CLERK     ¦ 1300 ¦ 1900 ¦ 950
MANAGER   ¦ 2450 ¦ 2975 ¦ 2850
PRESIDENT ¦ 5000 ¦ NULL ¦ NULL
SALESMAN  ¦ NULL ¦ NULL ¦ 5600
(5개 행)
```

SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)
```
-- 오라클 UNPIVOT: D10, D20, D30 열을 (부서, 급여) 행으로
SELECT * FROM PIVOT_RESULT
UNPIVOT (SAL FOR DEPTNO IN (D10 AS 10, D20 AS 20, D30 AS 30));
```

TIP PIVOT 은 FOR 절에 지정하지 않은 칼럼(위의 JOB)을 기준으로 **자동으로 GROUP BY** 해요. 그래서 인라인 뷰로 필요한 칼럼만 남겨 두는 것이 중요합니다. UNPIVOT 은 NULL 인 값을 기본으로 **빼고** 보여 줘요(INCLUDE NULLS 로 포함 가능).

확인 문제

Q1. 행으로 된 값을 열로 바꾸는 구문은?

① PIVOT
② UNPIVOT
③ ROLLUP
④ CUBE

Q2. PIVOT 에서 FOR 절과 집계 함수에 쓰이지 않은 칼럼은 어떻게 되나요?

① 무시된다
② 자동으로 GROUP BY 기준이 된다
③ 오류가 난다
④ 마지막 열이 된다

04.8 정규 표현식

정규 표현식은 문자열의 **패턴**을 표현하는 방법이에요. 오라클은 REGEXP_ 로 시작하는 함수를 제공해요.

패턴	뜻	패턴	뜻	
.	아무 글자 하나	^ / $	문자열 시작 / 끝	
* / + / ?	0번 이상 / 1번 이상 / 0~1번	{n} {n,m}	n번 / n~m번	
[abc] / [^abc]	괄호 안 글자 하나 / 제외	₩d / ₩w / ₩s	숫자 / 문자·숫자·_ / 공백	
a₩	b	a 또는 b	( )	묶음(그룹)

함수	하는 일
REGEXP_LIKE(문자열, 패턴)	패턴과 맞으면 참 (WHERE 절 조건)
REGEXP_SUBSTR(문자열, 패턴)	패턴에 맞는 부분 꺼내기
REGEXP_REPLACE(문자열, 패턴, 바꿀값)	패턴에 맞는 부분 바꾸기
REGEXP_INSTR(문자열, 패턴)	패턴 위치
REGEXP_COUNT(문자열, 패턴)	패턴 개수

SQL
```
-- 이름이 A 또는 S 로 시작하고 N 으로 끝나지 않는 사원 (오라클: WHERE REGEXP_LIKE(ENAME, '^[AS]'))
SELECT ENAME FROM EMP WHERE REGEXP_LIKE(ENAME, '^[AS]') AND NOT REGEXP_LIKE(ENAME, 'N$');
```

실행 결과
```
ENAME
-----
SMITH
SCOTT
ADAMS
(3개 행)
```

SQL
```
SELECT REGEXP_SUBSTR('주문번호 A-2026-0042', '[0-9]+') AS 처음숫자,
       REGEXP_REPLACE('010-1234-5678', '[0-9]{4}$', '****') AS 가리기,
       REGEXP_COUNT('banana', 'an') AS 개수,
       REGEXP_INSTR('SQL Expert', '[A-Z][a-z]') AS 위치
FROM DUAL;
```

실행 결과

```
처음숫자 ¦ 가리기        ¦ 개수 ¦ 위치
---------+---------------+------+-----
2026     ¦ 010-1234-**** ¦ 2    ¦ 5
(1개 행)
```

확인 문제

Q1. 정규 표현식 '^S' 의 뜻은?

① S 로 시작한다
② S 로 끝난다
③ S 를 포함하지 않는다
④ S 가 한 번 이상 나온다

Q2. 패턴에 맞는 행만 고르는 WHERE 절 조건에 쓰는 오라클 함수는?

① REGEXP_SUBSTR
② REGEXP_LIKE
③ REGEXP_REPLACE
④ REGEXP_COUNT

Q3. REGEXP_COUNT('banana', 'a') 의 결과는?

① 1
② 2
③ 3
④ 6

CHAPTER 05

[2과목] 관리 구문

학습 목표

- INSERT, UPDATE, DELETE, MERGE 로 데이터를 바꿀 수 있다.
- COMMIT, ROLLBACK, SAVEPOINT 로 트랜잭션을 제어한다.
- CREATE, ALTER, DROP, TRUNCATE 와 제약조건을 안다.
- DELETE, TRUNCATE, DROP 의 차이를 설명한다.
- GRANT, REVOKE 와 ROLE 로 권한을 관리하는 법을 안다.

05.1 DML

DML 은 테이블의 **데이터를 바꾸는** 명령이에요. (SELECT 를 포함하기도 해요)

SQL
```
-- INSERT: 칼럼을 지정해서 / 모든 칼럼 순서대로
INSERT INTO DEPT (DEPTNO, DNAME, LOC) VALUES (50, 'DESIGN', 'SEOUL');
INSERT INTO DEPT VALUES (60, 'QA', 'BUSAN');
SELECT * FROM DEPT WHERE DEPTNO >= 50;
```

실행 결과
```
1개 행이 삽입되었습니다.

1개 행이 삽입되었습니다.

DEPTNO ¦ DNAME  ¦ LOC
-------+--------+------
50     ¦ DESIGN ¦ SEOUL
60     ¦ QA     ¦ BUSAN
(2개 행)
```

SQL
```
-- 서브쿼리로 여러 행을 한 번에 INSERT
CREATE TABLE EMP_SALES AS SELECT * FROM EMP WHERE 1 = 2;     -- 구조만 복사 (행 없음)
INSERT INTO EMP_SALES SELECT * FROM EMP WHERE JOB = 'SALESMAN';
SELECT EMPNO, ENAME, JOB FROM EMP_SALES;
```

실행 결과
```
생성되었습니다.

4개 행이 삽입되었습니다.

EMPNO ¦ ENAME  ¦ JOB
------+--------+---------
7499  ¦ ALLEN  ¦ SALESMAN
7521  ¦ WARD   ¦ SALESMAN
7654  ¦ MARTIN ¦ SALESMAN
7844  ¦ TURNER ¦ SALESMAN
(4개 행)
```

SQL
```
-- UPDATE: WHERE 로 고칠 행을 정한다 (WHERE 가 없으면 모든 행!)
UPDATE EMP SET SAL = SAL * 1.1 WHERE DEPTNO = 10;
SELECT ENAME, SAL FROM EMP WHERE DEPTNO = 10;
```

실행 결과
```
3개 행이 수정되었습니다.

ENAME  ¦ SAL
-------+-----
CLARK  ¦ 2695
KING   ¦ 5500
MILLER ¦ 1430
(3개 행)
```

SQL
```
-- DELETE: WHERE 로 지울 행을 정한다
DELETE FROM EMP WHERE ENAME = 'MILLER';
SELECT COUNT(*) AS 남은인원 FROM EMP;
```

실행 결과
```
1개 행이 삭제되었습니다.

남은인원
--------
13
(1개 행)
```

MERGE: 있으면 UPDATE, 없으면 INSERT

SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)
```
MERGE INTO DEPT D
USING (SELECT 40 AS DEPTNO, 'OPS' AS DNAME, 'SEOUL' AS LOC FROM DUAL
       UNION ALL SELECT 70, 'AI', 'PANGYO' FROM DUAL) S
ON (D.DEPTNO = S.DEPTNO)
WHEN MATCHED THEN UPDATE SET D.DNAME = S.DNAME, D.LOC = S.LOC          -- 40번은 있으니 수정
WHEN NOT MATCHED THEN INSERT (DEPTNO, DNAME, LOC) VALUES (S.DEPTNO, S.DNAME, S.LOC);  -- 70번은 새로
```

SQL
```
-- 위 MERGE 와 같은 결과 (SQLite 의 UPSERT 문법)
INSERT INTO DEPT (DEPTNO, DNAME, LOC) VALUES (40, 'OPS', 'SEOUL'), (70, 'AI', 'PANGYO')
ON CONFLICT (DEPTNO) DO UPDATE SET DNAME = excluded.DNAME, LOC = excluded.LOC;
SELECT * FROM DEPT WHERE DEPTNO IN (40, 70);
```

실행 결과
```
2개 행이 삽입되었습니다.

DEPTNO ¦ DNAME ¦ LOC
-------+-------+-------
40     ¦ OPS   ¦ SEOUL
70     ¦ AI    ¦ PANGYO
(2개 행)
```

TIP 오라클에서 DML 은 **자동 커밋되지 않아요.** COMMIT 해야 확정되고, 다른 사용자도 볼 수 있어요. (SQL Server 는 기본이 자동 커밋)

확인 문제

Q1. WHERE 절 없이 UPDATE EMP SET SAL = 0 을 실행하면?

① 오류가 난다
② 첫 행만 바뀐다
③ 모든 행의 SAL 이 0 이 된다
④ 아무 행도 바뀌지 않는다

Q2. 조건에 맞으면 UPDATE, 없으면 INSERT 하는 DML 은?

① UPSERT
② MERGE
③ REPLACE
④ UNION

Q3. CREATE TABLE T AS SELECT * FROM EMP WHERE 1 = 2 의 결과는?

① EMP 의 모든 행이 복사된다
② 구조만 복사되고 행은 없다
③ 오류가 난다
④ 제약조건까지 모두 복사된다

05.2 TCL

TCL 은 트랜잭션을 **확정하거나 취소**하는 명령이에요.

명령	하는 일
COMMIT	지금까지의 변경을 **확정** (영구 저장)
ROLLBACK	마지막 COMMIT 이후 변경을 **취소**
SAVEPOINT 이름	중간 저장점 만들기
ROLLBACK TO 이름	저장점까지만 되돌리기

```sql
BEGIN;                                          -- (SQLite) 트랜잭션 시작. 오라클은 첫 DML 에서 자동 시작
UPDATE EMP SET SAL = 9999 WHERE ENAME = 'SMITH';
SAVEPOINT SP1;
DELETE FROM EMP WHERE ENAME = 'SMITH';
ROLLBACK TO SP1;                                -- DELETE 만 취소
SELECT ENAME, SAL FROM EMP WHERE ENAME = 'SMITH';
ROLLBACK;                                       -- UPDATE 까지 모두 취소
SELECT ENAME, SAL FROM EMP WHERE ENAME = 'SMITH';
```

실행 결과

```
트랜잭션을 시작합니다.

1개 행이 수정되었습니다.

저장점이 만들어졌습니다.

1개 행이 삭제되었습니다.

롤백되었습니다.

ENAME ¦ SAL
------+-----
SMITH ¦ 9999
(1개 행)

롤백되었습니다.

ENAME ¦ SAL
------+----
SMITH ¦ 800
(1개 행)
```

TIP **오라클 자동 커밋**: DDL(CREATE, ALTER, DROP, TRUNCATE)이나 DCL 을 실행하면 그 전의 DML 이 **자동으로 커밋**돼요. 정상 종료 시에도 커밋, 비정상 종료 시에는 롤백됩니다. SQL Server 는 DDL 도 트랜잭션 안에서 롤백할 수 있어요.

확인 문제

Q1. 마지막 COMMIT 이후의 변경을 모두 취소하는 명령은?

① COMMIT
② ROLLBACK
③ SAVEPOINT
④ REVOKE

Q2. 오라클에서 UPDATE 후 CREATE TABLE 을 실행하고 ROLLBACK 하면 UPDATE 는?

① 취소된다
② 이미 커밋되어 취소되지 않는다
③ 오류가 난다
④ 절반만 취소된다

Q3. SAVEPOINT A 다음에 DELETE, 그 뒤 ROLLBACK TO A 를 하면?

① DELETE 만 취소된다
② 모든 변경이 취소된다
③ 아무것도 취소되지 않는다
④ 트랜잭션이 커밋된다

05.3 DDL

DDL 은 테이블 같은 **데이터베이스 객체의 구조**를 만들고 바꾸는 명령이에요.

SQL

```
CREATE TABLE MEMBER (
  MEMBER_ID   INTEGER      PRIMARY KEY,                -- 기본키: NOT NULL + UNIQUE
  EMAIL       VARCHAR(50)  NOT NULL UNIQUE,            -- 비어 있으면 안 되고 중복 불가
  AGE         INTEGER      CHECK (AGE >= 0),           -- 값의 범위 검사
  GRADE       VARCHAR(10)  DEFAULT 'BASIC',            -- 값을 안 주면 기본값
  DEPTNO      INTEGER      REFERENCES DEPT(DEPTNO)     -- 외래키
);
INSERT INTO MEMBER (MEMBER_ID, EMAIL, AGE) VALUES (1, 'kim@ex.com', 20);
SELECT * FROM MEMBER;
```

실행 결과

```
생성되었습니다.

1개 행이 삽입되었습니다.

MEMBER_ID ¦ EMAIL      ¦ AGE ¦ GRADE ¦ DEPTNO
----------+------------+-----+-------+-------
1         ¦ kim@ex.com ¦ 20  ¦ BASIC ¦ NULL
(1개 행)
```

제약조건

제약조건	뜻
PRIMARY KEY	기본키. **NOT NULL + UNIQUE**, 테이블당 하나
UNIQUE	중복 불가 (**NULL 은 허용**)
NOT NULL	NULL 불가
CHECK	조건을 만족하는 값만 허용
FOREIGN KEY	다른 테이블의 PK 참조, 참조 무결성 보장
DEFAULT	값이 없을 때 기본값 (엄밀히는 제약조건이 아님)

SQL (오류가 나는 것이 정상인 예)

```
INSERT INTO MEMBER (MEMBER_ID, EMAIL, AGE) VALUES (2, 'lee@ex.com', -5);   -- CHECK 위반
```

실행 결과

```
오류: CHECK constraint failed: AGE >= 0
```

외래키 삭제 옵션

옵션	부모 행을 지우면
(기본, RESTRICT / NO ACTION)	자식이 있으면 **삭제 불가**
ON DELETE CASCADE	자식 행도 **함께 삭제**
ON DELETE SET NULL	자식의 FK 를 **NULL 로**

ALTER, RENAME, DROP, TRUNCATE

SQL

```
ALTER TABLE MEMBER ADD PHONE VARCHAR(20);              -- 칼럼 추가
ALTER TABLE MEMBER RENAME COLUMN PHONE TO TEL;          -- 칼럼 이름 변경
ALTER TABLE MEMBER RENAME TO CUSTOMER;                  -- 테이블 이름 변경
SELECT * FROM CUSTOMER;
```

실행 결과

```
변경되었습니다.

변경되었습니다.

변경되었습니다.

MEMBER_ID ¦ EMAIL      ¦ AGE ¦ GRADE ¦ DEPTNO ¦ TEL
----------+------------+-----+-------+--------+-----
1         ¦ kim@ex.com ¦ 20  ¦ BASIC ¦ NULL   ¦ NULL
(1개 행)
```

SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)

```
-- 오라클 전용 / 표준 문법 (이 앱의 SQLite 는 일부 미지원)
ALTER TABLE CUSTOMER MODIFY (TEL VARCHAR(30) NOT NULL);   -- 칼럼 속성 변경 (SQL Server: ALTER COLUMN)
ALTER TABLE CUSTOMER DROP COLUMN TEL;                     -- 칼럼 삭제
ALTER TABLE CUSTOMER ADD CONSTRAINT CK_AGE CHECK (AGE < 150);
TRUNCATE TABLE CUSTOMER;                                  -- 모든 행 삭제 (구조는 남김)
DROP TABLE CUSTOMER;                                      -- 테이블 자체 삭제
```

구분	DELETE	TRUNCATE	DROP
종류	DML	DDL	DDL
지우는 것	행 (WHERE 가능)	**모든 행**	**테이블 자체** (구조까지)
롤백	가능	불가 (자동 커밋)	불가 (자동 커밋)
저장 공간	그대로 유지	초기화 (반납)	모두 반납
속도	느림 (한 행씩 로그)	빠름	빠름

확인 문제

Q1. PRIMARY KEY 제약조건의 성질은?

① NULL 허용 + 중복 불가
② NOT NULL + UNIQUE
③ 테이블당 여러 개 가능
④ 중복 허용

Q2. UNIQUE 제약조건이 있는 칼럼에 대한 설명으로 옳은 것은?

① NULL 값을 넣을 수 있다
② NULL 을 넣을 수 없다
③ 테이블당 하나만 만들 수 있다
④ 기본키와 완전히 같다

Q3. 부모 행을 지우면 자식 행도 함께 지워지게 하는 옵션은?

① ON DELETE SET NULL
② ON DELETE CASCADE
③ ON DELETE RESTRICT
④ ON UPDATE CASCADE

Q4. 테이블 구조는 남기고 모든 행을 빠르게 지우며 롤백할 수 없는 명령은?

① DELETE
② TRUNCATE
③ DROP
④ REMOVE

Q5. DELETE, TRUNCATE, DROP 중 DML 은?

① DELETE
② TRUNCATE
③ DROP
④ 모두 DDL

05.4 DCL

DCL 은 사용자에게 **권한을 주고 빼앗는** 명령이에요.

```
SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)
CREATE USER SCOTT2 IDENTIFIED BY tiger;            -- 사용자 만들기 (오라클)
GRANT CREATE SESSION, CREATE TABLE TO SCOTT2;      -- 시스템 권한: 접속, 테이블 생성
GRANT SELECT, INSERT ON EMP TO SCOTT2;             -- 객체 권한: EMP 테이블 조회·입력
GRANT SELECT ON EMP TO SCOTT2 WITH GRANT OPTION;   -- 받은 권한을 다른 사람에게 줄 수 있게
REVOKE INSERT ON EMP FROM SCOTT2;                  -- 권한 회수
```

구분	예
시스템 권한	CREATE SESSION, CREATE TABLE, CREATE USER …
객체 권한	특정 객체에 대한 SELECT, INSERT, UPDATE, DELETE, ALTER, REFERENCES …

WITH GRANT OPTION vs WITH ADMIN OPTION

옵션	대상	회수할 때
WITH GRANT OPTION	**객체 권한**	A → B → C 로 준 뒤 A 가 B 의 권한을 회수하면 **C 의 권한도 함께 회수**
WITH ADMIN OPTION	**시스템 권한**	B 의 권한을 회수해도 **C 의 권한은 남음**

ROLE: 권한 묶음

권한을 사용자마다 하나씩 주면 관리가 힘들어요. **ROLE** 에 권한을 모아 두고 ROLE 을 부여하면 편합니다.

```
SQL (오라클 문법 · 예제 DB 에서 실행하지 않음)
CREATE ROLE APP_USER;
GRANT CREATE SESSION, SELECT ANY TABLE TO APP_USER;
GRANT APP_USER TO SCOTT2, KIM;                     -- 여러 사용자에게 한 번에
```

TIP 오라클이 미리 만들어 둔 ROLE: **CONNECT**(접속 관련), **RESOURCE**(테이블 등 객체 생성 관련), **DBA**(모든 권한).

확인 문제

Q1. 사용자에게 권한을 부여하는 명령은?

① GRANT
② REVOKE
③ COMMIT
④ ALLOW

Q2. A 가 B 에게 WITH GRANT OPTION 으로 객체 권한을 주고 B 가 C 에게 준 뒤, A 가 B 의 권한을 회수하면 C 는?

① 권한이 그대로 남는다
② 권한이 함께 회수된다
③ 권한이 두 배가 된다
④ 오류가 난다

Q3. 여러 권한을 묶어 한 번에 부여하기 위한 것은?

① VIEW
② ROLE
③ SCHEMA
④ SYNONYM

Q4. 시스템 권한을 다른 사용자에게 다시 줄 수 있게 하는 옵션은?

① WITH GRANT OPTION
② WITH ADMIN OPTION
③ WITH ROLE OPTION
④ WITH CHECK OPTION

CHAPTER 06

기출공략집

학습 목표

- 시험에 자주 나오는 대표 유형 30가지를 익힌다.
- 기출 유형을 바꾼 변형문제 2회분으로 실전 감각을 기른다.
- 실전 모의고사(앱)로 시간 안에 50문항을 푸는 연습을 한다.

06.0 시험 안내와 활용법

항목	내용
과목	1과목 데이터 모델링의 이해 (10문항), 2과목 SQL 기본 및 활용 (40문항)
시간	90분, 50문항 (객관식 4지선다)
합격 기준	총점 **60점 이상** + 과목별 **40% 이상** (과락 주의)

TIP 대표 유형 30선 → 변형문제 01·02회 → 앱의 **실전 모의고사 01~05회** (과목 비율대로 50문항, 90분 타이머, 과락까지 채점) 순서로 풀어 보세요. 틀린 문제는 해설의 장·절로 돌아가 다시 읽는 것이 가장 빠른 길이에요. 시험 출제 범위와 비율은 바뀔 수 있으니 시험 전에 한국데이터산업진흥원 공지를 꼭 확인하세요.

06.1 대표 유형 30선

문제

Q1. [유형 1 모델링 단계] 추상화 수준이 가장 높고, 업무 중심의 핵심 엔터티와 관계를 정의하는 단계는?

① 개념적 모델링
② 논리적 모델링
③ 물리적 모델링
④ 정규화 모델링

Q2. [유형 2 3층 스키마] 데이터가 실제로 저장되는 방식(파일, 인덱스)을 표현하는 스키마는?

① 외부 스키마
② 개념 스키마
③ 내부 스키마
④ 논리 스키마

Q3. [유형 3 엔터티 분류] 고객, 상품처럼 다른 엔터티로부터 영향을 받지 않고 독립적으로 생성되는 엔터티는?

① 기본 엔터티
② 중심 엔터티
③ 행위 엔터티
④ 사건 엔터티

Q4. [유형 4 속성] 원래 값이 바뀔 때마다 함께 갱신해야 해서 정합성 관리가 필요한 속성은?

① 기본 속성
② 설계 속성
③ 파생 속성
④ 복합 속성

Q5. [유형 5 관계 표기] IE 표기법에서 관계선 끝의 동그라미(○)가 뜻하는 것은?

① 1 (하나)
② 다수 (Many)
③ 선택 참여 (0 가능)
④ 필수 참여

Q6. [유형 6 식별자 관계] 부모 엔터티의 주식별자를 자식 엔터티의 주식별자 일부로 사용하는 관계는?

① 식별자 관계
② 비식별자 관계
③ 순환 관계
④ 상호배타 관계

Q7. [유형 7 정규화] 주문상세(주문번호, 상품번호, 상품명, 수량) PK = (주문번호, 상품번호) 이고 상품번호 → 상품명 일 때 위반한 정규형은?

① 1정규형
② 2정규형
③ 3정규형
④ BCNF

Q8. [유형 8 NULL 과 집계] 칼럼 C 의 값이 (10, 20, NULL, 30) 일 때 SUM(C)/COUNT(*) 와 AVG(C) 는?

① 20, 20
② 15, 20
③ 15, 15
④ NULL, 20

Q9. [유형 9 SQL 분류] 다음 중 DDL 에 해당하는 것은?

① DELETE
② MERGE
③ TRUNCATE
④ REVOKE

Q10. [유형 10 실행 순서] SELECT 문의 논리적 실행 순서에서 HAVING 바로 다음에 실행되는 것은?

① WHERE
② GROUP BY
③ SELECT
④ ORDER BY

Q11. [유형 11 문자 함수] 오라클에서 LENGTH(LTRIM(' ab ')) 의 결과는?

① 2
② 4
③ 6
④ 8

Q12. [유형 12 숫자 함수] TRUNC(15.79, -1) 의 결과는?

① 10
② 15
③ 15.7
④ 20

Q13. [유형 13 CASE] SAL = 800 일 때 CASE WHEN SAL > 2000 THEN 'H' WHEN SAL > 1000 THEN 'M' END 의 결과는?

① H
② M
③ NULL
④ 오류

Q14. [유형 14 NULL 함수] COALESCE(NULL, NVL(NULL, 3), 5) 의 결과는?

① NULL
② 3
③ 5
④ 8

Q15. [유형 15 연산자 우선순위] WHERE A = 1 OR B = 2 AND C = 3 과 같은 조건은?

① (A = 1 OR B = 2) AND C = 3
② A = 1 OR (B = 2 AND C = 3)
③ A = 1 AND B = 2 OR C = 3
④ 모두 같다

Q16. [유형 16 LIKE ESCAPE] 이름에 밑줄(_) 문자 자체가 들어 있는 행을 찾는 조건은?

① NAME LIKE '%_%'
② NAME LIKE '%#_%' ESCAPE '#'
③ NAME = '_'
④ NAME LIKE '_'

Q17. [유형 17 HAVING] 예제 DB 에서 SELECT DEPTNO FROM EMP GROUP BY DEPTNO HAVING COUNT(*) >= 5 의 결과 행 수는?

① 1
② 2
③ 3
④ 4

Q18. [유형 18 COUNT 와 NULL] 예제 DB 에서 SELECT COUNT(COMM) FROM EMP 의 결과는?

① 4
② 10
③ 14
④ 0

Q19. [유형 19 NULL 정렬] 오라클에서 ORDER BY COMM DESC 로 정렬하면 첫 행의 COMM 은?

① NULL
② 1400
③ 0
④ 300

Q20. [유형 20 조인 건수] 예제 DB 에서 SELECT * FROM DEPT D LEFT OUTER JOIN EMP E ON D.DEPTNO = E.DEPTNO 의 결과 행 수는?

① 4
② 14
③ 15
④ 56

Q21. [유형 21 USING] 다음 중 오류가 나는 SQL 은?

① SELECT DEPTNO, ENAME FROM EMP JOIN DEPT USING (DEPTNO)
② SELECT E.DEPTNO, ENAME FROM EMP E JOIN DEPT D USING (DEPTNO)
③ SELECT DNAME FROM EMP NATURAL JOIN DEPT
④ SELECT ENAME FROM EMP E JOIN DEPT D ON E.DEPTNO = D.DEPTNO

Q22. [유형 22 오라클 외부 조인] WHERE D.DEPTNO = E.DEPTNO(+) 와 같은 표준 조인은?

① FROM DEPT D LEFT OUTER JOIN EMP E ON D.DEPTNO = E.DEPTNO
② FROM DEPT D RIGHT OUTER JOIN EMP E ON D.DEPTNO = E.DEPTNO
③ FROM DEPT D INNER JOIN EMP E ON D.DEPTNO = E.DEPTNO
④ FROM DEPT D FULL OUTER JOIN EMP E ON D.DEPTNO = E.DEPTNO

Q23. [유형 23 서브쿼리] 단일행 비교 연산자(=) 뒤의 서브쿼리가 여러 행을 돌려주면?

① 첫 행만 비교한다
② 마지막 행만 비교한다
③ 오류가 난다
④ IN 처럼 동작한다

Q24. [유형 24 NOT EXISTS] 예제 DB 에서 다음 SQL 의 결과 행 수는?

```
SQL
SELECT DNAME FROM DEPT D
WHERE NOT EXISTS (SELECT 1 FROM EMP E WHERE E.DEPTNO = D.DEPTNO);
```

① 1
② 3
③ 4
④ 0

Q25. [유형 25 집합 연산] SELECT DEPTNO FROM EMP UNION SELECT DEPTNO FROM DEPT 의 결과 행 수는?

① 18
② 14
③ 4
④ 3

Q26. [유형 26 ROLLUP] 예제 DB 에서 SELECT DEPTNO, SUM(SAL) FROM EMP GROUP BY ROLLUP(DEPTNO) 의 결과 행 수는?

① 3
② 4
③ 5
④ 6

Q27. [유형 27 순위 함수] 급여가 (300, 200, 200, 100) 일 때 RANK, DENSE_RANK 로 구한 100 의 순위는? (높은 급여가 1위)

① 4, 3
② 3, 3
③ 4, 4
④ 3, 4

Q28. [유형 28 TOP-N] 오라클에서 급여 상위 3명을 바르게 구하는 SQL 은?

① SELECT * FROM EMP WHERE ROWNUM <= 3 ORDER BY SAL DESC
② SELECT * FROM (SELECT * FROM EMP ORDER BY SAL DESC) WHERE ROWNUM <= 3
③ SELECT * FROM EMP WHERE ROWNUM BETWEEN 1 AND 3 ORDER BY SAL
④ SELECT TOP 3 * FROM EMP ORDER BY SAL DESC

Q29. [유형 29 계층형 질의] 예제 DB 에서 다음 SQL 의 결과 행 수는?

```
SQL
SELECT ENAME FROM EMP
START WITH ENAME = 'SMITH'
CONNECT BY PRIOR MGR = EMPNO;
```

① 1
② 3
③ 4
④ 14

Q30. [유형 30 자동 커밋] 오라클에서 UPDATE 를 실행한 뒤 CREATE TABLE 을 실행하고 ROLLBACK 했다. 올바른 설명은?

① UPDATE 와 CREATE 모두 취소된다
② UPDATE 만 취소된다
③ UPDATE 는 DDL 실행 때 자동 커밋되어 취소되지 않는다
④ ROLLBACK 이 오류를 낸다

06.2 기출 변형문제 01회

문제

Q1. 데이터 모델링의 3가지 관점이 아닌 것은?

① 데이터 관점
② 프로세스 관점
③ 데이터와 프로세스의 상관 관점
④ 사용자 인터페이스 관점

Q2. 엔터티의 특징으로 옳지 않은 것은?

① 업무에서 관리가 필요한 정보이다
② 인스턴스가 두 개 이상이다
③ 다른 엔터티와 관계가 전혀 없어도 된다
④ 속성을 가진다

Q3. 주식별자를 지정할 때 고려할 사항으로 거리가 먼 것은?

① 자주 바뀌지 않는 값을 고른다
② 이름처럼 중복될 수 있는 값은 피한다
③ 반드시 값이 있는 속성을 고른다
④ 가능한 많은 속성을 묶어 복합식별자로 만든다

Q4. 반정규화 기법 중 칼럼 반정규화에 해당하지 않는 것은?

① 중복 칼럼 추가
② 파생 칼럼 추가
③ 이력 테이블 칼럼 추가
④ 테이블 수평 분할

Q5. SQL 명령어와 분류의 연결이 틀린 것은?

① SELECT - DML
② ALTER - DDL
③ ROLLBACK - DCL
④ GRANT - DCL

Q6. 다음 SQL 의 결과는?

```sql
SELECT SUBSTR('SQLDEVELOPER', -4, 3) FROM DUAL;
```

① DEV
② OPE
③ PER
④ LOP

Q7. 오라클에서 '' (빈 문자열)에 대한 설명으로 옳은 것은?

① NULL 로 취급된다
② 공백 한 칸으로 취급된다
③ 0 으로 취급된다
④ 오류가 난다

Q8. 예제 DB 에서 SELECT COUNT(DISTINCT JOB) FROM EMP 의 결과는?

① 4
② 5
③ 14
④ 3

Q9. WHERE 절에 쓸 수 없는 것은?

① BETWEEN
② IN
③ IS NULL
④ SUM(SAL) > 1000

Q10. 다음 중 결과가 다른 하나는?

① NVL(COMM, 0)
② COALESCE(COMM, 0)
③ CASE WHEN COMM IS NULL THEN 0 ELSE COMM END
④ NULLIF(COMM, 0)

Q11. 예제 DB 에서 SELECT JOB, COUNT(*) FROM EMP GROUP BY JOB HAVING COUNT(*) > 3 의 결과 행 수는?

① 1
② 2
③ 3
④ 5

Q12. ORDER BY 에 대한 설명으로 옳은 것은?

① GROUP BY 를 쓰면 ORDER BY 를 쓸 수 없다
② SELECT 절에 없는 칼럼으로도 정렬할 수 있다 (GROUP BY 가 없을 때)
③ 칼럼 위치 번호는 쓸 수 없다
④ DESC 가 기본값이다

Q13. 예제 DB 에서 SELECT COUNT(*) FROM EMP E, SALGRADE S WHERE E.SAL BETWEEN S.LOSAL AND S.HISAL 의 결과는?

① 5
② 14
③ 70
④ 0

Q14. CROSS JOIN 에 대한 설명으로 옳은 것은?

① 조인 조건이 반드시 필요하다
② 두 테이블 행의 모든 조합(카티션 곱)을 만든다
③ 같은 이름 칼럼으로 조인한다
④ 외부 조인의 한 종류다

Q15. 다음 중 다중행 서브쿼리 연산자가 아닌 것은?

① IN
② ANY
③ ALL
④ =

Q16. 스칼라 서브쿼리에 대한 설명으로 옳은 것은?

① 한 행, 한 칼럼만 돌려줘야 한다
② FROM 절에만 쓸 수 있다
③ 여러 행을 돌려줄 수 있다
④ 메인쿼리 칼럼을 쓸 수 없다

Q17. INTERSECT 의 결과는?

① 두 결과의 합집합
② 두 결과에 모두 있는 행 (교집합)
③ 첫 결과에만 있는 행
④ 두 결과의 카티션 곱

Q18. GROUP BY CUBE(A, B) 가 만드는 그룹 조합의 수는?

① 2
② 3
③ 4
④ 6

Q19. 윈도우 함수에서 PARTITION BY 의 역할은?

① 정렬 기준을 정한다
② 계산할 그룹(파티션)을 나눈다
③ 행 수를 줄인다
④ 중복을 제거한다

Q20. 다음 SQL 에서 KING(5000) 의 결과 값은? (부서 10: MILLER 1300, CLARK 2450, KING 5000)

```sql
SELECT ENAME, SUM(SAL) OVER (PARTITION BY DEPTNO ORDER BY SAL) AS S
FROM EMP WHERE DEPTNO = 10;
```

① 5000
② 6300
③ 8750
④ 3750

Q21. 셀프 조인에 대한 설명으로 옳은 것은?

① 같은 테이블에 서로 다른 별칭을 붙여 조인한다
② 반드시 외부 조인이어야 한다
③ 별칭을 쓰지 않는다
④ 서로 다른 두 테이블만 가능하다

Q22. PIVOT 절에 대한 설명으로 옳지 않은 것은?

① 행 데이터를 열로 바꾼다
② 집계 함수와 FOR 절을 쓴다
③ GROUP BY 를 반드시 직접 써야 한다
④ IN 절에 열로 만들 값을 지정한다

Q23. DELETE 와 TRUNCATE 에 대한 설명으로 옳은 것은?

① 둘 다 DML 이다
② DELETE 는 롤백할 수 있고 TRUNCATE 는 오라클에서 롤백할 수 없다
③ TRUNCATE 는 WHERE 절을 쓸 수 있다
④ DELETE 는 테이블 구조까지 지운다

Q24. FOREIGN KEY 에 ON DELETE SET NULL 옵션을 주면 부모 행을 삭제할 때?

① 자식 행도 삭제된다
② 자식 행의 외래키 값이 NULL 이 된다
③ 삭제가 거부된다
④ 부모 행만 남는다

06.3 기출 변형문제 02회

문제

Q1. 데이터 독립성에 대한 설명으로 옳은 것은?

① 논리적 독립성은 개념 스키마가 바뀌어도 외부 스키마가 영향을 받지 않는 것이다
② 물리적 독립성은 외부 스키마가 바뀌어도 내부 스키마가 영향을 받지 않는 것이다
③ 3층 스키마는 외부·내부·물리 스키마다
④ 데이터 독립성은 중복을 늘린다

Q2. 다음 중 M:N 관계를 해소하기 위해 만드는 것은?

① 슈퍼타입
② 교차(연관) 엔터티
③ 보조식별자
④ 파생 속성

Q3. 비식별자 관계를 선택하기에 알맞은 경우는?

① 부모가 반드시 있어야만 자식이 생길 때
② 자식의 주식별자를 부모와 독립적으로 구성하고 싶을 때
③ 부모 키로 자식을 바로 조회해야 할 때
④ 관계가 매우 강할 때

Q4. 트랜잭션의 성질과 설명이 틀린 것은?

① 원자성 - 모두 실행되거나 모두 취소된다
② 일관성 - 실행 전후 일관된 상태를 유지한다
③ 고립성 - 커밋된 결과는 영구히 저장된다
④ 지속성 - 커밋된 결과는 장애가 나도 유지된다

Q5. 다음 SQL 의 결과는?

```SQL
SELECT ROUND(1234.567, 1), TRUNC(1234.567, -2), MOD(17, 5) FROM DUAL;
```

① 1234.6, 1200, 2
② 1234.5, 1200, 3
③ 1234.6, 1300, 2
④ 1235, 1234, 2

Q6. DECODE(DEPTNO, 10, 'A', 20, 'B') 에서 DEPTNO 가 30 이면?

① A
② B
③ NULL
④ 30

Q7. 예제 DB 에서 SELECT ENAME FROM EMP WHERE ENAME LIKE '%A%' AND SAL > 1500 의 결과 행 수는?

① 2
② 3
③ 4
④ 5

Q8. 다음 중 집계 함수에 대한 설명으로 틀린 것은?

① COUNT(*) 는 NULL 인 행도 센다
② SUM 은 NULL 을 빼고 더한다
③ AVG 는 NULL 을 0 으로 보고 평균을 낸다
④ MAX 는 NULL 을 무시한다

Q9. 다음 SQL 이 오류가 나는 이유는?

```sql
SELECT DEPTNO, ENAME, SUM(SAL) FROM EMP GROUP BY DEPTNO;
```

① SUM 을 쓸 수 없어서
② ENAME 이 GROUP BY 에 없고 집계 함수도 아니어서
③ DEPTNO 가 PK 가 아니어서
④ HAVING 이 없어서

Q10. 표준 조인에서 ON 절과 WHERE 절에 대한 설명으로 옳은 것은?

① ON 에는 조인 조건, WHERE 에는 일반 검색 조건을 주로 쓴다
② NATURAL JOIN 에도 ON 을 쓸 수 있다
③ USING 과 ON 을 함께 쓴다
④ WHERE 를 쓰면 ON 을 생략해야 한다

Q11. 예제 DB 에서 SELECT COUNT(*) FROM EMP E RIGHT OUTER JOIN DEPT D ON E.DEPTNO = D.DEPTNO 의 결과는?

① 14
② 15
③ 4
④ 18

Q12. 다음 중 연관 서브쿼리를 사용한 것은?

① WHERE SAL > (SELECT AVG(SAL) FROM EMP)
② WHERE SAL > (SELECT AVG(SAL) FROM EMP X WHERE X.DEPTNO = E.DEPTNO)
③ WHERE DEPTNO IN (SELECT DEPTNO FROM DEPT)
④ FROM (SELECT * FROM EMP)

Q13. 예제 DB 에서 다음 SQL 의 결과 행 수는?

```sql
SELECT JOB FROM EMP WHERE DEPTNO = 20
MINUS
SELECT JOB FROM EMP WHERE DEPTNO = 30;
```

① 1
② 2
③ 3
④ 0

Q14. GROUPING SETS 에 대한 설명으로 옳은 것은?

① 전체 합계를 항상 만든다
② 지정한 그룹 기준별 집계를 각각 구하고, 순서가 결과 집합에 영향을 주지 않는다
③ ROLLUP 과 결과가 항상 같다
④ 칼럼이 하나일 때만 쓸 수 있다

Q15. 급여 (400, 300, 300, 200, 100) 에 ROW_NUMBER, RANK, DENSE_RANK 를 각각 매길 때 200 의 값은? (내림차순)

① 4, 3, 3
② 4, 4, 3
③ 3, 3, 3
④ 4, 4, 4

Q16. LAG(SAL, 2) OVER (ORDER BY HIREDATE) 의 뜻은?

① 입사일 순으로 2행 앞(이전)의 급여
② 2행 뒤(다음)의 급여
③ 급여의 2배
④ 상위 2명의 급여

Q17. SQL Server 에서 상위 5건을 동점까지 포함해 조회하는 구문은?

① SELECT TOP(5) ...
② SELECT TOP(5) WITH TIES ... ORDER BY ...
③ WHERE ROWNUM <= 5
④ LIMIT 5

Q18. 계층형 질의에서 START WITH 절의 역할은?

① 정렬 기준을 정한다
② 계층 전개를 시작할 루트 행을 정한다
③ 부모와 자식을 잇는 조건을 정한다
④ 리프 노드를 표시한다

Q19. 정규 표현식 '[0-9]{3}-[0-9]{4}' 와 일치하는 문자열은?

① 12-3456
② 123-4567
③ 1234-567
④ abc-defg

Q20. INSERT INTO T SELECT * FROM S 에 대한 설명으로 옳은 것은?

① 테이블 T 를 새로 만든다
② S 의 조회 결과를 이미 있는 테이블 T 에 넣는다
③ VALUES 를 함께 써야 한다
④ 한 행만 넣을 수 있다

Q21. 다음 중 오라클에서 자동 커밋이 일어나지 않는 것은?

① CREATE TABLE 실행
② DROP TABLE 실행
③ GRANT 실행
④ UPDATE 실행

Q22. 칼럼 이름을 바꾸는 표준 SQL 은?

① ALTER TABLE T MODIFY A TO B
② ALTER TABLE T RENAME COLUMN A TO B
③ RENAME COLUMN A TO B
④ UPDATE T SET A = B

Q23. WITH ADMIN OPTION 으로 받은 시스템 권한을 다른 사용자에게 준 뒤, 처음 권한을 회수하면?

① 연쇄적으로 모두 회수된다
② 다시 받은 사용자의 권한은 남는다
③ 권한을 준 사람이 오류를 받는다
④ 모든 사용자의 권한이 사라진다

Q24. 뷰(View)의 장점이 아닌 것은?

① 복잡한 질의를 단순하게 만든다
② 필요한 칼럼만 보여 줘 보안에 유리하다
③ 테이블 구조가 바뀌어도 응용 프로그램 영향이 적다
④ 데이터를 따로 저장해 조회가 항상 빨라진다

CHAPTER 07

보너스 합격자료집

학습 목표

- 고난이도 예상문제로 헷갈리기 쉬운 함정을 정리한다.
- 예제 DB 로 SQL 을 직접 작성하는 실습 문제를 푼다.

07.1 SQLD 고난이도 예상문제

문제

Q1. 예제 DB 에서 다음 SQL 의 결과는?

```SQL
SELECT COUNT(*) FROM EMP WHERE COMM NOT IN (300, 500, NULL);
```

① 2
② 8
③ 0
④ 12

Q2. 예제 DB 에서 다음 SQL 의 결과는?

```SQL
SELECT COUNT(*) FROM EMP WHERE COMM IN (300, 500, NULL);
```

① 0
② 2
③ 3
④ 12

Q3. 다음 두 SQL 의 결과 행 수는? (예제 DB)

```sql
SELECT D.DEPTNO, E.ENAME FROM DEPT D LEFT JOIN EMP E ON D.DEPTNO = E.DEPTNO AND E.SAL > 3000;
SELECT D.DEPTNO, E.ENAME FROM DEPT D LEFT JOIN EMP E ON D.DEPTNO = E.DEPTNO WHERE E.SAL > 3000;
```

① 4행, 1행
② 1행, 1행
③ 4행, 4행
④ 15행, 1행

Q4. 예제 DB 에서 다음 SQL 의 결과는?

```sql
SELECT COUNT(*) FROM EMP WHERE SAL > ALL (SELECT SAL FROM EMP WHERE DEPTNO = 40);
```

① 0
② 14
③ 1
④ 오류

Q5. 다음 중 결과가 나머지와 다른 것은? (COMM 은 NULL 이 있는 칼럼)

① SELECT SUM(COMM) FROM EMP
② SELECT SUM(NVL(COMM, 0)) FROM EMP
③ SELECT SUM(COMM + 0) FROM EMP WHERE COMM IS NULL
④ SELECT SUM(COMM) FROM EMP WHERE COMM IS NOT NULL

Q6. 예제 DB 에서 다음 SQL 의 결과 행 수는?

```SQL
SELECT DEPTNO, JOB, SUM(SAL) FROM EMP
GROUP BY GROUPING SETS ((DEPTNO, JOB), ());
```

① 9
② 10
③ 13
④ 14

Q7. 예제 DB 에서 다음 SQL 의 결과는?

```
SQL
SELECT COUNT(*) FROM EMP E1
WHERE 3 > (SELECT COUNT(*) FROM EMP E2 WHERE E2.SAL > E1.SAL);
```

① 3
② 4
③ 5
④ 2

Q8. ROWNUM 에 대한 다음 SQL 중 결과가 한 건 이상 나오는 것은? (오라클)

① SELECT * FROM EMP WHERE ROWNUM < 3
② SELECT * FROM EMP WHERE ROWNUM = 2
③ SELECT * FROM EMP WHERE ROWNUM > 1
④ SELECT * FROM EMP WHERE ROWNUM BETWEEN 2 AND 3

Q9. 윈도우 함수에서 ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING 의 뜻은?

① 처음부터 현재 행까지
② 현재 행과 바로 앞·뒤 한 행씩, 최대 3행
③ 앞의 모든 행
④ 현재 행만

Q10. 오라클 계층형 질의에서 WHERE 절의 조건은 언제 적용되나요?

① 계층 전개 전에 START WITH 대상만 거른다
② 계층 전개가 모두 끝난 뒤 결과 행을 거른다
③ CONNECT BY 조건과 똑같이 동작한다
④ 루트 행에만 적용된다

Q11. UNION 으로 합친 결과를 정렬할 때 올바른 것은?

① 각 SELECT 마다 ORDER BY 를 쓴다
② 마지막 SELECT 뒤에 ORDER BY 를 한 번 쓴다
③ 첫 SELECT 에만 ORDER BY 를 쓴다
④ UNION 결과는 정렬할 수 없다

Q12. TRUNCATE 와 DELETE(조건 없음)에 대한 설명으로 틀린 것은?

① 둘 다 모든 행을 지운다
② TRUNCATE 가 보통 더 빠르다
③ 둘 다 오라클에서 롤백할 수 있다
④ TRUNCATE 는 저장 공간을 반납한다

07.2 SQL 실습 문제

예제 DB(EMP, DEPT, SALGRADE)로 직접 SQL 을 써 보는 문제예요. 앱에서는 SQL 을 입력하고 **[채점]** 을 누르면 정답 SQL 의 결과와 비교해 맞았는지 알려 줘요. (행의 순서는 정답에 ORDER BY 가 있을 때만 비교해요)

1. 급여 3000 이상

급여(SAL)가 3000 이상인 사원의 이름(ENAME)과 급여를 조회하세요.

2. 커미션을 받는 사원

커미션(COMM)이 NULL 이 아닌 사원의 이름, 급여, 커미션을 조회하세요.

3. 연봉 계산

모든 사원의 이름과 연봉(SAL * 12 + 커미션, 커미션이 없으면 0으로)을 연봉이 높은 순으로 조회하세요. 연봉 칼럼 이름은 연봉 으로 하세요.

4. 이름 패턴

이름이 S 로 시작하거나 이름에 'AR' 이 들어가는 사원의 이름을 조회하세요.

5. 부서별 인원과 평균

부서번호별 사원 수와 평균 급여(소수 첫째 자리까지 반올림)를 부서번호 순으로 조회하세요.

6. 업무별 최고 급여 (HAVING)

업무(JOB)별 최고 급여를 구하되, 최고 급여가 2000 이상인 업무만 조회하세요.

7. 사원과 부서 이름

사원 이름과 그 사원의 부서 이름(DNAME), 근무지(LOC)를 조회하세요.

8. 사원이 없는 부서까지

모든 부서의 부서번호, 부서 이름, 사원 수를 부서번호 순으로 조회하세요. 사원이 없는 부서는 0명으로 나와야 해요.

9. 급여 등급

사원 이름, 급여, 급여 등급(SALGRADE 의 GRADE)을 조회하세요.

10. 관리자 이름

사원 이름과 그 관리자의 이름을 조회하세요. 관리자가 없는 사원(KING)도 나와야 하고, 관리자 칼럼 이름은 관리자 로 하세요.

11. 평균보다 많이

전체 평균 급여보다 급여가 많은 사원의 이름과 급여를 조회하세요. (서브쿼리)

12. 부서 최고 급여자

각 부서에서 급여가 가장 높은 사원의 부서번호, 이름, 급여를 조회하세요.

13. 부서 안 급여 순위

사원 이름, 부서번호, 급여, 부서 안에서의 급여 순위(RANK, 높은 급여가 1위)를 부서번호·순위 순으로 조회하세요. 순위 칼럼 이름은 순위 로 하세요.

14. 급여 상위 5명

급여가 높은 순으로 5명의 이름과 급여를 조회하세요. 급여가 같으면 이름순으로 하세요.

15. 업무별 부서 급여표 (PIVOT 흉내)

업무별로 10, 20, 30번 부서의 급여 합계를 열(D10, D20, D30)로 보여 주세요. 업무 이름 순으로 정렬하세요.

SQL 실습 문제 정답

1. 급여 3000 이상

정답 SQL

```
SELECT ENAME, SAL FROM EMP WHERE SAL >= 3000;
```

실행 결과

```
ENAME ¦ SAL
------+-----
SCOTT ¦ 3000
KING  ¦ 5000
FORD  ¦ 3000
(3개 행)
```

2. 커미션을 받는 사원

정답 SQL

```
SELECT ENAME, SAL, COMM FROM EMP WHERE COMM IS NOT NULL;
```

실행 결과

```
ENAME  ¦ SAL  ¦ COMM
-------+------+-----
ALLEN  ¦ 1600 ¦ 300
WARD   ¦ 1250 ¦ 500
MARTIN ¦ 1250 ¦ 1400
TURNER ¦ 1500 ¦ 0
(4개 행)
```

3. 연봉 계산

정답 SQL

```
SELECT ENAME, SAL * 12 + NVL(COMM, 0) AS 연봉 FROM EMP ORDER BY 연봉 DESC, ENAME;
```

실행 결과

```
ENAME  ¦ 연봉
-------+------
KING   ¦ 60000
FORD   ¦ 36000
SCOTT  ¦ 36000
JONES  ¦ 35700
BLAKE  ¦ 34200
CLARK  ¦ 29400
ALLEN  ¦ 19500
TURNER ¦ 18000
MARTIN ¦ 16400
MILLER ¦ 15600
WARD   ¦ 15500
ADAMS  ¦ 13200
JAMES  ¦ 11400
SMITH  ¦ 9600
(14개 행)
```

4. 이름 패턴

정답 SQL

```
SELECT ENAME FROM EMP WHERE ENAME LIKE 'S%' OR ENAME LIKE '%AR%';
```

실행 결과

```
ENAME
------
SMITH
WARD
MARTIN
CLARK
SCOTT
(5개 행)
```

5. 부서별 인원과 평균

정답 SQL

```
SELECT DEPTNO, COUNT(*) AS 인원, ROUND(AVG(SAL), 1) AS 평균 FROM EMP GROUP BY DEPTNO ORDER BY DEPTNO;
```

실행 결과

```
DEPTNO ¦ 인원 ¦ 평균
-------+------+-------
10     ¦ 3    ¦ 2916.7
20     ¦ 5    ¦ 2175
30     ¦ 6    ¦ 1566.7
(3개 행)
```

6. 업무별 최고 급여 (HAVING)

정답 SQL

```
SELECT JOB, MAX(SAL) AS 최고 FROM EMP GROUP BY JOB HAVING MAX(SAL) >= 2000;
```

실행 결과

```
JOB       ¦ 최고
----------+-----
ANALYST   ¦ 3000
MANAGER   ¦ 2975
PRESIDENT ¦ 5000
(3개 행)
```

7. 사원과 부서 이름

정답 SQL

```
SELECT E.ENAME, D.DNAME, D.LOC FROM EMP E JOIN DEPT D ON E.DEPTNO = D.DEPTNO;
```

실행 결과

```
ENAME  ¦ DNAME      ¦ LOC
-------+------------+---------
SMITH  ¦ RESEARCH   ¦ DALLAS
ALLEN  ¦ SALES      ¦ CHICAGO
WARD   ¦ SALES      ¦ CHICAGO
JONES  ¦ RESEARCH   ¦ DALLAS
MARTIN ¦ SALES      ¦ CHICAGO
BLAKE  ¦ SALES      ¦ CHICAGO
CLARK  ¦ ACCOUNTING ¦ NEW YORK
SCOTT  ¦ RESEARCH   ¦ DALLAS
KING   ¦ ACCOUNTING ¦ NEW YORK
TURNER ¦ SALES      ¦ CHICAGO
ADAMS  ¦ RESEARCH   ¦ DALLAS
JAMES  ¦ SALES      ¦ CHICAGO
FORD   ¦ RESEARCH   ¦ DALLAS
MILLER ¦ ACCOUNTING ¦ NEW YORK
(14개 행)
```

8. 사원이 없는 부서까지

정답 SQL

```
SELECT D.DEPTNO, D.DNAME, COUNT(E.EMPNO) AS 인원 FROM DEPT D LEFT JOIN EMP E ON D.DEPTNO = E.DEPTNO GROUP BY
D.DEPTNO, D.DNAME ORDER BY D.DEPTNO;
```

실행 결과

```
DEPTNO ¦ DNAME      ¦ 인원
-------+------------+-----
10     ¦ ACCOUNTING ¦ 3
20     ¦ RESEARCH   ¦ 5
30     ¦ SALES      ¦ 6
40     ¦ OPERATIONS ¦ 0
(4개 행)
```

9. 급여 등급

정답 SQL

```
SELECT E.ENAME, E.SAL, S.GRADE FROM EMP E JOIN SALGRADE S ON E.SAL BETWEEN S.LOSAL AND S.HISAL;
```

실행 결과

```
ENAME  ¦ SAL  ¦ GRADE
-------+------+------
SMITH  ¦ 800  ¦ 1
ALLEN  ¦ 1600 ¦ 3
WARD   ¦ 1250 ¦ 2
JONES  ¦ 2975 ¦ 4
MARTIN ¦ 1250 ¦ 2
BLAKE  ¦ 2850 ¦ 4
CLARK  ¦ 2450 ¦ 4
SCOTT  ¦ 3000 ¦ 4
KING   ¦ 5000 ¦ 5
TURNER ¦ 1500 ¦ 3
ADAMS  ¦ 1100 ¦ 1
JAMES  ¦ 950  ¦ 1
FORD   ¦ 3000 ¦ 4
MILLER ¦ 1300 ¦ 2
(14개 행)
```

10. 관리자 이름

정답 SQL

```
SELECT W.ENAME, M.ENAME AS 관리자 FROM EMP W LEFT JOIN EMP M ON W.MGR = M.EMPNO;
```

실행 결과

```
ENAME  ¦ 관리자
-------+-------
SMITH  ¦ FORD
ALLEN  ¦ BLAKE
WARD   ¦ BLAKE
JONES  ¦ KING
MARTIN ¦ BLAKE
BLAKE  ¦ KING
CLARK  ¦ KING
SCOTT  ¦ JONES
KING   ¦ NULL
TURNER ¦ BLAKE
ADAMS  ¦ SCOTT
JAMES  ¦ BLAKE
FORD   ¦ JONES
MILLER ¦ CLARK
(14개 행)
```

11. 평균보다 많이

정답 SQL

```
SELECT ENAME, SAL FROM EMP WHERE SAL > (SELECT AVG(SAL) FROM EMP);
```

실행 결과

```
ENAME ¦ SAL
------+-----
JONES ¦ 2975
BLAKE ¦ 2850
CLARK ¦ 2450
SCOTT ¦ 3000
KING  ¦ 5000
FORD  ¦ 3000
(6개 행)
```

12. 부서 최고 급여자

정답 SQL

```
SELECT DEPTNO, ENAME, SAL FROM EMP E WHERE SAL = (SELECT MAX(SAL) FROM EMP WHERE DEPTNO = E.DEPTNO);
```

실행 결과

```
DEPTNO ¦ ENAME ¦ SAL
-------+-------+-----
30     ¦ BLAKE ¦ 2850
20     ¦ SCOTT ¦ 3000
10     ¦ KING  ¦ 5000
20     ¦ FORD  ¦ 3000
(4개 행)
```

13. 부서 안 급여 순위

정답 SQL

```
SELECT ENAME, DEPTNO, SAL, RANK() OVER (PARTITION BY DEPTNO ORDER BY SAL DESC) AS 순위 FROM EMP ORDER BY DEPTNO,
순위, ENAME;
```

실행 결과

```
ENAME  ¦ DEPTNO ¦ SAL  ¦ 순위
-------+--------+------+-----
KING   ¦ 10     ¦ 5000 ¦ 1
CLARK  ¦ 10     ¦ 2450 ¦ 2
MILLER ¦ 10     ¦ 1300 ¦ 3
FORD   ¦ 20     ¦ 3000 ¦ 1
SCOTT  ¦ 20     ¦ 3000 ¦ 1
JONES  ¦ 20     ¦ 2975 ¦ 3
ADAMS  ¦ 20     ¦ 1100 ¦ 4
SMITH  ¦ 20     ¦ 800  ¦ 5
BLAKE  ¦ 30     ¦ 2850 ¦ 1
ALLEN  ¦ 30     ¦ 1600 ¦ 2
TURNER ¦ 30     ¦ 1500 ¦ 3
MARTIN ¦ 30     ¦ 1250 ¦ 4
WARD   ¦ 30     ¦ 1250 ¦ 4
JAMES  ¦ 30     ¦ 950  ¦ 6
(14개 행)
```

14. 급여 상위 5명

정답 SQL

```
SELECT ENAME, SAL FROM EMP ORDER BY SAL DESC, ENAME LIMIT 5;
```

실행 결과

```
ENAME ¦ SAL
------+-----
KING  ¦ 5000
FORD  ¦ 3000
SCOTT ¦ 3000
JONES ¦ 2975
BLAKE ¦ 2850
(5개 행)
```

15. 업무별 부서 급여표 (PIVOT 흉내)

정답 SQL

```
SELECT JOB, SUM(CASE WHEN DEPTNO = 10 THEN SAL END) AS D10, SUM(CASE WHEN DEPTNO = 20 THEN SAL END) AS D20,
SUM(CASE WHEN DEPTNO = 30 THEN SAL END) AS D30 FROM EMP GROUP BY JOB ORDER BY JOB;
```

실행 결과

```
JOB       ¦ D10  ¦ D20  ¦ D30
----------+------+------+-----
ANALYST   ¦ NULL ¦ 6000 ¦ NULL
CLERK     ¦ 1300 ¦ 1900 ¦ 950
MANAGER   ¦ 2450 ¦ 2975 ¦ 2850
PRESIDENT ¦ 5000 ¦ NULL ¦ NULL
SALESMAN  ¦ NULL ¦ NULL ¦ 5600
(5개 행)
```

부록 예제 데이터베이스

교안의 모든 SQL 예제와 앱의 실습 문제가 쓰는 데이터예요. 결과 행 수를 묻는 문제를 풀 때 이 표를 보며 직접 세어 보세요. 테이블을 만드는 SQL 은 저장소의 guide/sample_db.sql 에 있어요.

DEPT

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

EMP

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	800	NULL	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
7566	JONES	MANAGER	7839	1981-04-02	2975	NULL	20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250	1400	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850	NULL	30
7782	CLARK	MANAGER	7839	1981-06-09	2450	NULL	10
7788	SCOTT	ANALYST	7566	1987-04-19	3000	NULL	20
7839	KING	PRESIDENT	NULL	1981-11-17	5000	NULL	10
7844	TURNER	SALESMAN	7698	1981-09-08	1500	0	30
7876	ADAMS	CLERK	7788	1987-05-23	1100	NULL	20
7900	JAMES	CLERK	7698	1981-12-03	950	NULL	30
7902	FORD	ANALYST	7566	1981-12-03	3000	NULL	20
7934	MILLER	CLERK	7782	1982-01-23	1300	NULL	10

SALGRADE

GRADE	LOSAL	HISAL
1	700	1200
2	1201	1400
3	1401	2000
4	2001	3000
5	3001	9999

정답과 해설

01.1 데이터 모델의 이해

번호	정답	해설
Q1	④	모델링의 3가지 특징은 추상화·단순화·명확화예요.
Q2	②	논리적 모델링에서 키·속성·관계를 정의하고 정규화해요.
Q3	②	저장 구조(내부)와 전체 구조(개념)가 분리된 것이 물리적 독립성이에요.
Q4	①	외부 스키마는 사용자마다 필요한 데이터를 보여 주는 관점(뷰)이에요.

01.2 엔터티

번호	정답	해설
Q1	③	엔터티는 인스턴스가 2개 이상이어야 해요.
Q2	③	업무 활동으로 생기는 엔터티는 사건 엔터티예요.
Q3	①	기본(키) 엔터티는 다른 엔터티에 의존하지 않고 생겨요. (사원, 부서, 고객)

01.3 속성

번호	정답	해설
Q1	③	계산으로 얻는 값은 파생 속성이에요.
Q2	②	도메인은 자료형·길이·허용 값 등 값의 범위예요.
Q3	②	식별을 위해 새로 만든 코드·번호는 설계 속성이에요.

01.4 관계

번호	정답	해설
Q1	④	관계명·차수(카디널리티)·선택사양(선택성)이 3요소예요.
Q2	②	교차(연관) 엔터티를 두어 1:M, M:1 로 해소해요.
Q3	②	업무 행위로 연결되는 관계는 행위 관계예요.

01.5 식별자

번호	정답	해설
Q1	④	주식별자의 특징은 유일성·최소성·불변성·존재성이에요.
Q2	②	비식별자 관계에서는 부모 키가 자식의 일반 속성(FK)이 되고 점선으로 표시해요.
Q3	②	다른 엔터티에서 온 식별자(FK)는 외부식별자예요.
Q4	④	주식별자는 NULL 이 될 수 없어요(존재성).

02.1 정규화

번호	정답	해설
Q1	②	부분 함수 종속을 없애는 것이 2정규형이에요.
Q2	③	키가 아닌 속성을 거쳐 종속되는 이행 함수 종속을 없애는 것이 3정규형이에요.
Q3	②	반정규화는 성능을 위한 의도적 중복이며, 다른 방법을 먼저 검토해요.
Q4	③	원하지 않는 정보까지 함께 지워지는 것이 삭제 이상이에요.

02.2 관계와 조인의 이해

번호	정답	해설
Q1	②	같은 엔터티끼리의 관계를 순환(재귀, 자기 참조) 관계라고 해요.
Q2	①	둘 중 하나와만 연결되는 관계는 상호배타 관계예요.
Q3	②	자식의 FK 와 부서의 PK 를 같게 놓아 관계를 이어요.

02.3 모델이 표현하는 트랜잭션의 이해

번호	정답	해설
Q1	①	All or Nothing 은 원자성(Atomicity)이에요.
Q2	③	완료된 결과가 영구히 남는 것이 지속성이에요.
Q3	②	함께 발생해야 하는 엔터티는 필수 관계로 표현해 하나의 트랜잭션으로 처리해요.

02.4 NULL 속성의 이해

번호	정답	해설
Q1	③	NULL 과의 산술 연산 결과는 NULL 이에요.
Q2	②	NULL 비교는 IS NULL 을 써요. = NULL 은 결과가 없어요.
Q3	②	집계 함수는 NULL 을 빼고 (100 + 200) / 2 = 150 이에요.
Q4	②	COUNT(*) 는 모든 행, COUNT(칼럼) 은 NULL 이 아닌 행만 세요.

02.5 본질식별자 vs 인조식별자

번호	정답	해설
Q1	②	일련번호 PK 만으로는 같은 업무 데이터의 중복을 막지 못해요.
Q2	①	업무 의미가 있는 원래 값으로 만든 식별자는 본질식별자예요.

03.1 관계형 데이터베이스 개요

번호	정답	해설
Q1	④	GRANT 는 권한을 주는 DCL 이에요.
Q2	④	트랜잭션을 제어하는 TCL 이에요.
Q3	②	외래키(FK)는 다른 테이블의 PK 를 참조해 관계를 만들어요.

03.2 SELECT 문

번호	정답	해설
Q1	②	FROM 부터 실행하고 SELECT 는 거의 마지막, ORDER BY 가 맨 끝이에요.
Q2	④	ORDER BY 는 SELECT 다음에 실행되어 별칭을 쓸 수 있어요.
Q3	②	SELECT DISTINCT 로 중복을 제거해요.

03.3 함수(Function)

번호	정답	해설
Q1	②	3번째 글자 T 부터 4글자: TABA
Q2	③	-1 은 일의 자리에서 반올림해 50 이에요.

번호	정답	해설
Q3	②	첫 인수가 NULL 이면 세 번째 값 B 를 돌려줘요.
Q4	②	처음으로 NULL 이 아닌 값 10 이에요.
Q5	①	두 값이 같으면 NULL 을 돌려줘요.

03.4 WHERE 절

번호	정답	해설
Q1	②	BETWEEN 은 양 끝 값을 포함해요.
Q2	②	두 번째 글자가 A 여야 해요: W-A-RD
Q3	②	NOT, AND, OR 순서로 계산해요.
Q4	②	NOT IN 목록에 NULL 이 있으면 모든 비교가 UNKNOWN 이 되어 결과가 없어요.

03.5 GROUP BY, HAVING 절

번호	정답	해설
Q1	②	그룹 조건은 HAVING 에 써요. WHERE 에는 집계 함수를 못 써요.
Q2	②	COUNT 만 0 이고 SUM·AVG 등은 NULL 이에요.
Q3	④	그룹마다 값이 여러 개인 칼럼(ENAME)은 집계 없이 쓸 수 없어요.

03.6 ORDER BY 절

번호	정답	해설
Q1	④	ORDER BY 는 가장 마지막에 실행돼요.
Q2	②	오라클은 NULL 을 가장 큰 값으로 취급해 오름차순에서 맨 뒤에 와요.

03.7 조인

번호	정답	해설
Q1	②	BETWEEN, > 등으로 잇는 조인은 비등가 조인이에요.
Q2	②	N 개 테이블이면 최소 N-1 개예요.
Q3	③	모든 조합이 나오는 카티션 곱: 14 × 4 = 56

03.8 표준 조인

번호	정답	해설
Q1	②	NATURAL JOIN 은 같은 이름 칼럼 전부로 조인하고 ON 을 쓸 수 없어요.
Q2	②	USING 에 쓴 칼럼에는 테이블 별칭을 붙일 수 없어요.
Q3	①	왼쪽(DEPT)의 행을 모두 남기는 LEFT OUTER JOIN 이에요.
Q4	②	NULL 로 채워진 행이 하나 있어서 COUNT(*) 는 1, COUNT(E.EMPNO) 는 0 이에요.

04.1 서브쿼리

번호	정답	해설
Q1	②	FROM 절의 서브쿼리를 인라인 뷰라고 해요.
Q2	③	ALL 은 모든 값보다 커야 하므로 최댓값 3000 보다 커야 해요.

번호	정답	해설
Q3	①	ANY 는 하나라도 보다 크면 되므로 최솟값 1000 보다 크면 돼요.
Q4	②	연관(상관) 서브쿼리는 메인쿼리 행마다 다시 실행돼요.

04.2 집합 연산자

번호	정답	해설
Q1	②	UNION ALL 은 중복 제거·정렬을 하지 않아요.
Q2	③	차집합은 표준·SQL Server 에서 EXCEPT 예요.
Q3	①	첫 번째 쿼리의 칼럼 이름(별칭)을 따라요.

04.3 그룹 함수

번호	정답	해설
Q1	④	ROLLUP 은 오른쪽부터 줄여 가며 (A,B) → (A) → () 만 만들어요. B 별 소계는 CUBE 에 있어요.
Q2	③	GROUPING SETS(A, B) 는 A 별, B 별 집계만 만들어요.
Q3	③	CUBE 는 모든 조합 $2^3 = 8$ 가지예요.
Q4	②	집계로 만들어진 NULL 이면 1, 원래 값이면 0 이에요.

04.4 윈도우 함수

번호	정답	해설
Q1	②	DENSE_RANK 는 순위를 건너뛰지 않아 1, 2, 2, 3 이에요.
Q2	①	RANK 는 같은 순위 다음을 건너뛰어 1, 2, 2, 4 예요.
Q3	①	LAG 는 앞(이전) 행, LEAD 는 뒤(다음) 행의 값이에요.
Q4	②	처음부터 현재 행(같은 값 포함)까지의 RANGE 예요.
Q5	③	ROW_NUMBER 는 1, 2, 3… 고유한 번호를 붙여요.

04.5 TOP-N 쿼리

번호	정답	해설
Q1	②	ROWNUM 은 1번부터 차례로 매겨져서 1번이 없으면 2번도 없어요.
Q2	②	SQL Server 는 SELECT TOP(3) 을 써요.
Q3	②	ROWNUM 이 정렬 전에 매겨지므로 정렬을 먼저 인라인 뷰에서 해요.

04.6 계층형 질의와 셀프 조인

번호	정답	해설
Q1	①	PRIOR 쪽(EMPNO)이 이전 행(부모)이라 위에서 아래로 내려가요.
Q2	②	루트가 LEVEL 1, 그 자식이 2 예요.
Q3	②	ORDER SIBLINGS BY 는 계층 구조를 유지하며 형제끼리 정렬해요.
Q4	②	CONNECT_BY_ISLEAF 는 리프 노드에서 1 이에요.

04.7 PIVOT 절과 UNPIVOT 절

번호	정답	해설
Q1	①	PIVOT 은 행 → 열, UNPIVOT 은 열 → 행이에요.
Q2	②	남은 칼럼 기준으로 자동 그룹화돼서, 인라인 뷰로 칼럼을 골라 두는 게 좋아요.

04.8 정규 표현식

번호	정답	해설
Q1	①	^ 는 문자열의 시작이에요. 끝은 $ 예요.
Q2	②	REGEXP_LIKE 는 참·거짓 조건이라 WHERE 절에 써요.
Q3	③	banana 에 a 가 3번 나와요.

05.1 DML

번호	정답	해설
Q1	③	WHERE 가 없으면 테이블의 모든 행이 대상이에요.
Q2	②	오라클·표준의 MERGE 문이에요.
Q3	②	조건이 항상 거짓이라 행 없이 칼럼 구조만 만들어져요. (NOT NULL 외 제약조건은 복사 안 됨)

05.2 TCL

번호	정답	해설
Q1	②	ROLLBACK 은 확정되지 않은 변경을 모두 되돌려요.
Q2	②	DDL 실행 시 앞의 DML 이 자동 커밋돼요.
Q3	①	저장점 이후의 변경만 되돌려요.

05.3 DDL

번호	정답	해설
Q1	②	기본키는 NULL 이 안 되고 중복도 안 되며 테이블당 하나예요.
Q2	①	UNIQUE 는 중복만 막고 NULL 은 허용해요.
Q3	②	CASCADE 는 자식까지 연쇄 삭제해요.
Q4	②	TRUNCATE 는 DDL 이라 자동 커밋되고 구조는 남아요.
Q5	①	DELETE 만 DML 이라 롤백할 수 있어요.

05.4 DCL

번호	정답	해설
Q1	①	GRANT 로 주고 REVOKE 로 회수해요.
Q2	②	객체 권한(WITH GRANT OPTION)은 연쇄적으로 회수돼요.
Q3	②	ROLE 에 권한을 모아 사용자에게 부여해요.
Q4	②	시스템 권한은 WITH ADMIN OPTION, 객체 권한은 WITH GRANT OPTION 이에요.

06.1 대표 유형 30선

번호	정답	해설
Q1	①	개념 → 논리 → 물리 순으로 구체화돼요. 가장 추상적인 것은 개념적 모델링이에요. (01.1)

번호	정답	해설
Q2	③	물리적 저장 구조는 내부 스키마예요. (01.1)
Q3	①	독립적으로 생기는 키 엔터티가 기본 엔터티예요. (01.2)
Q4	③	계산으로 만든 파생 속성은 원본과 일치시키는 비용이 들어요. (01.3)
Q5	③	○ 는 선택(Optional), | 는 필수(Mandatory)예요. (01.4)
Q6	①	부모 키가 자식 PK 의 일부가 되면 식별자 관계(실선)예요. (01.5)
Q7	②	상품명이 PK 의 일부(상품번호)에만 종속되는 부분 함수 종속이라 2정규형 위반이에요. (02.1)
Q8	②	SUM 60 / COUNT(*) 4 = 15, AVG 는 NULL 을 빼고 60 / 3 = 20 이에요. (02.4)
Q9	③	TRUNCATE 는 행을 지우지만 DDL 이에요. (03.1)
Q10	③	FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY (03.2)
Q11	②	왼쪽 공백만 지워 'ab ' 가 되어 길이 4 예요. (03.3)
Q12	①	일의 자리 아래를 버려 10 이에요. ROUND(15.79, -1) 이면 20 이에요. (03.3)
Q13	③	맞는 조건이 없고 ELSE 도 없으면 NULL 이에요. (03.3)
Q14	②	NVL(NULL, 3) = 3 이 처음으로 NULL 이 아닌 값이에요. (03.3)
Q15	②	AND 가 OR 보다 먼저 계산돼요. (03.4)
Q16	②	_ 는 '아무 글자 하나'라서, 글자 그대로 찾으려면 ESCAPE 문자를 앞에 붙여요. (03.4)
Q17	②	10번 3명, 20번 5명, 30번 6명이라 20·30번 두 행이에요. (03.5)
Q18	①	COMM 이 NULL 이 아닌 사원은 4명(300, 500, 1400, 0)이에요. 0 도 값이에요. (02.4)
Q19	①	오라클은 NULL 을 가장 큰 값으로 보므로 내림차순에서 맨 앞이에요. (03.6)
Q20	③	사원 14명 + 사원이 없는 40번 부서 1행 = 15행이에요. (03.8)
Q21	②	USING 에 쓴 칼럼에는 테이블 별칭을 붙일 수 없어요. (03.8)
Q22	①	(+) 는 데이터가 모자란 쪽(EMP)에 붙여요. DEPT 를 모두 남기므로 LEFT OUTER JOIN 이에요. (03.7)
Q23	③	여러 행이면 IN, ANY, ALL 같은 다중행 연산자를 써야 해요. (04.1)
Q24	①	사원이 없는 부서는 40번(OPERATIONS) 하나예요. (04.1)
Q25	③	중복을 없애 10, 20, 30, 40 네 행이에요. UNION ALL 이면 18행이에요. (04.2)
Q26	②	부서 3개 행 + 전체 합계 1행 = 4행이에요. (04.3)
Q27	①	RANK 는 1, 2, 2, 4 / DENSE_RANK 는 1, 2, 2, 3 이에요. (04.4)
Q28	②	ROWNUM 은 정렬 전에 매겨지므로 인라인 뷰에서 먼저 정렬해요. TOP 은 SQL Server 문법이에요. (04.5)
Q29	③	역방향(자식 → 부모)으로 SMITH → FORD → JONES → KING, 4행이에요. (04.6)
Q30	③	DDL 을 실행하면 그 전의 DML 이 자동 커밋돼요. (05.2)

06.2 기출 변형문제 01회

번호	정답	해설
Q1	④	데이터·프로세스·상관(데이터와 프로세스) 관점이 3가지예요.
Q2	③	엔터티는 다른 엔터티와 최소 1개 이상 관계가 있어야 해요. (코드성 엔터티 등 예외는 있어요)
Q3	④	최소성: 유일성을 만족하는 최소한의 속성으로 구성해요.
Q4	④	테이블 분할은 테이블 반정규화예요.

번호	정답	해설
Q5	③	ROLLBACK 은 TCL 이에요.
Q6	②	시작 위치가 음수면 뒤에서부터 세요. 뒤에서 4번째 O 부터 3글자: OPE
Q7	①	오라클은 길이 0 인 문자열을 NULL 로 처리해요.
Q8	②	CLERK, SALESMAN, MANAGER, ANALYST, PRESIDENT 5가지예요.
Q9	④	집계 함수 조건은 HAVING 에 써요.
Q10	④	NULLIF(COMM, 0) 은 COMM 이 0 이면 NULL 을 돌려줘요. 나머지는 NULL 을 0 으로 바꿔요.
Q11	②	CLERK 4명, SALESMAN 4명만 3명보다 많아요.
Q12	②	일반 SELECT 에서는 SELECT 목록에 없는 칼럼으로도 정렬할 수 있어요.
Q13	②	모든 사원의 급여가 정확히 한 등급 범위에 들어가므로 14행이에요.
Q14	②	CROSS JOIN 은 M × N 행을 만들어요.
Q15	④	= 는 단일행 비교 연산자예요.
Q16	①	SELECT 절 등에서 값 하나처럼 쓰이므로 1행 1열이어야 해요.
Q17	②	INTERSECT 는 교집합이에요.
Q18	③	(A,B), (A), (B), () 네 가지예요.
Q19	②	파티션별로 따로 계산하되 행 수는 그대로예요.
Q20	③	급여 오름차순 누적합: 1300 → 3750 → 8750 이에요.
Q21	①	같은 테이블을 두 번 쓰려면 별칭으로 구분해요.
Q22	③	PIVOT 은 나머지 칼럼으로 자동 그룹화해요.
Q23	②	TRUNCATE 는 DDL 이라 자동 커밋돼요.
Q24	②	자식의 FK 를 NULL 로 바꿔요.

06.3 기출 변형문제 02회

번호	정답	해설
Q1	①	논리적 독립성: 개념 ↔ 외부, 물리적 독립성: 내부 ↔ 개념이에요.
Q2	②	교차 엔터티로 1:M, M:1 두 관계로 풀어요.
Q3	②	약한 관계이거나 자식 PK 를 독립적으로 두고 싶을 때 비식별자 관계를 써요.
Q4	③	'영구 저장'은 지속성이에요. 고립성은 다른 트랜잭션의 간섭을 받지 않는 것이에요.
Q5	①	반올림 1234.6, 백의 자리 아래 버림 1200, 17 ÷ 5 나머지 2
Q6	③	기본값을 생략하면 일치하는 값이 없을 때 NULL 이에요.
Q7	②	이름에 A 가 있는 사원 중 급여 1500 초과: ALLEN 1600, BLAKE 2850, CLARK 2450 — 3명이에요.
Q8	③	AVG 도 NULL 을 빼고 계산해요.
Q9	②	GROUP BY 를 쓰면 SELECT 에는 그룹 칼럼과 집계 함수만 올 수 있어요.
Q10	①	조인 조건과 검색 조건을 나눠 쓰면 읽기 쉬워요. NATURAL JOIN 은 ON 을 쓸 수 없어요.
Q11	②	DEPT 쪽을 모두 남기므로 사원 14행 + 40번 부서 1행 = 15행이에요.
Q12	②	메인쿼리 칼럼(E.DEPTNO)을 서브쿼리에서 쓰면 연관 서브쿼리예요.
Q13	①	20번 업무(CLERK, MANAGER, ANALYST) 중 30번에 없는 것은 ANALYST 하나예요.

번호	정답	해설
Q14	②	GROUPING SETS(A, B) 는 A 별, B 별 집계만 만들고 순서와 무관해요. () 를 넣으면 합계도 만들 수 있어요.
Q15	②	ROW_NUMBER 1~5 에서 4, RANK 1,2,2,4,5 에서 4, DENSE_RANK 1,2,2,3,4 에서 3 이에요.
Q16	①	LAG 는 앞, LEAD 는 뒤의 값을 가져와요.
Q17	②	WITH TIES 는 마지막 순위와 같은 값의 행까지 포함해요. ORDER BY 가 필요해요.
Q18	②	연결 조건은 CONNECT BY, 시작 행은 START WITH 예요.
Q19	②	숫자 3자리, 하이픈, 숫자 4자리예요.
Q20	②	서브쿼리 INSERT 는 여러 행을 한 번에 넣어요. 테이블을 만드는 것은 CREATE TABLE ... AS SELECT 예요.
Q21	④	DML(UPDATE)은 COMMIT 해야 확정돼요. DDL·DCL 은 자동 커밋을 일으켜요.
Q22	②	ALTER TABLE ... RENAME COLUMN ... TO ... 예요. (SQL Server 는 sp_rename)
Q23	②	시스템 권한(WITH ADMIN OPTION)은 연쇄 회수되지 않아요. 객체 권한(WITH GRANT OPTION)은 연쇄 회수돼요.
Q24	④	일반 뷰는 데이터를 저장하지 않는 가상 테이블이라 성능이 자동으로 좋아지지는 않아요.

07.1 SQLD 고난이도 예상문제

번호	정답	해설
Q1	③	NOT IN 목록에 NULL 이 있으면 모든 행의 조건이 UNKNOWN 이 되어 0 건이에요.
Q2	②	IN 은 OR 로 바뀌어 COMM = 300 OR COMM = 500 OR COMM = NULL. NULL 비교는 무시되고 2건이 나와요.
Q3	①	ON 의 조건은 조인할 짝만 거르고 DEPT 는 모두 남아 4행(10번만 KING, 나머지 NULL). WHERE 는 조인 뒤 거르므로 KING 1행이에요.
Q4	②	서브쿼리 결과가 비어 있으면 > ALL 은 참이 되어 모든 행(14건)이 나와요. > ANY 였다면 0건이에요.
Q5	③	NULL 인 행만 더하면 결과는 NULL 이에요. 나머지는 모두 2200 이에요.
Q6	②	(부서, 업무) 조합 9개(10번 3, 20번 3, 30번 3) + 전체 합계 1행 = 10행이에요.
Q7	①	나보다 급여가 높은 사원이 3명 미만인 사원을 세요. KING(0명), SCOTT(1명), FORD(1명)는 해당하고, JONES 는 위에 KING·SCOTT·FORD 3명이 있어 제외돼요. 동점자(SCOTT·FORD)가 있어 '상위 3명'이 정확히 3명으로 나오는 연관 서브쿼리 TOP-N 패턴이에요.
Q8	①	ROWNUM 은 1부터 순서대로 매겨져 1을 포함하지 않는 조건은 결과가 없어요.
Q9	②	이동 평균 같은 계산에 써요.
Q10	②	WHERE 는 전개 후 필터라서 중간 노드를 빼도 그 자식은 남아요. 가지째 빼려면 CONNECT BY 절에 조건을 둬요.
Q11	②	집합 연산 전체에 대해 맨 끝에 한 번만 써요.
Q12	③	TRUNCATE 는 DDL 이라 오라클에서 롤백할 수 없어요.